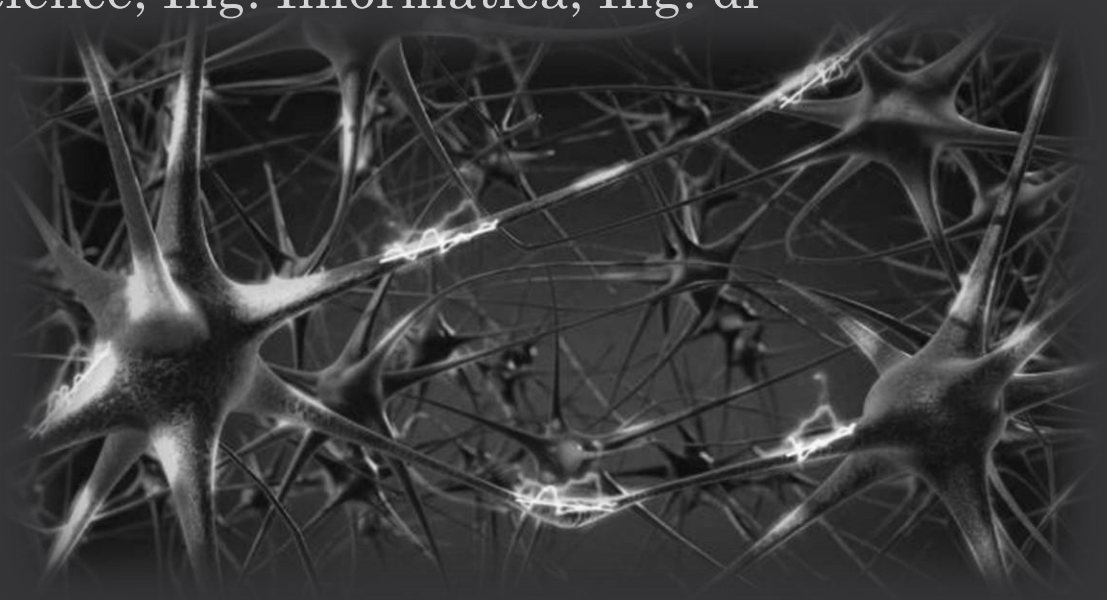
Intelligenza Artificiale

Introduzione alle Reti neurali

Corsi di Laurea in Informatica, Ing. Gestionale, Metodi Matemaici per la Data Science, Ing. Informatica, Ing. di Internet

(a.a. 2025-2026)

Roberto Basili



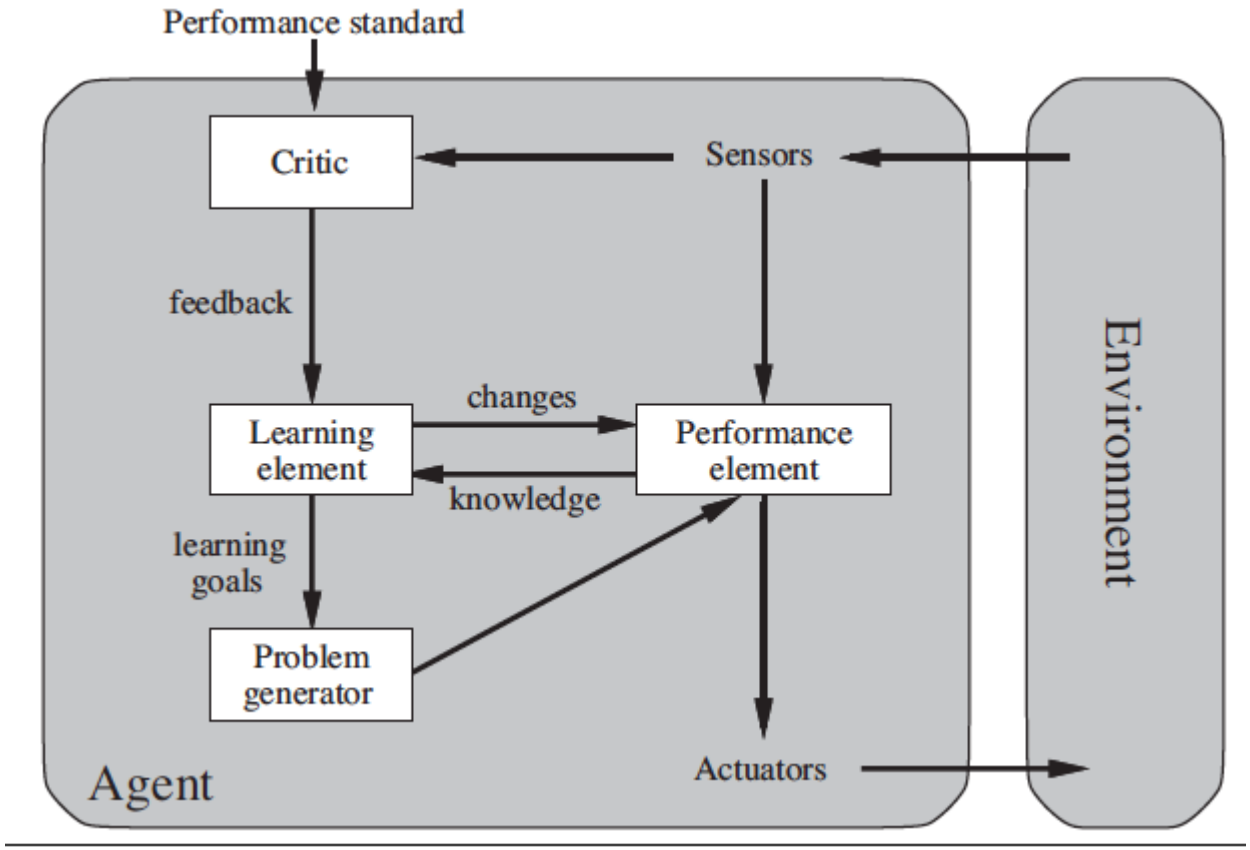
Overview (AIMA chpt. 18.6.1,18.6.3-4, 18.7)

- Recap
  - Agents & machine learning
  - Learning from examples:
    - The process of model selection
- Artificial Neural Networks
  - The linear regression problem: potential and limitations
  - Multilayer Perceptrons
  - Training MLPs
- NN Applications: image recognition
- NN Applications: NLP
- Advanced Architectures

Introduction to machine learning

- Introduction to machine learning, recap
  - Task definition, Learning methodology, Learning issues: representing hypothesis
  - The Model Selection process
  - Learning paradigms
    - Supervised learning
    - Unsupervised learning
    - Reinforcement learning

AIMA learning architecture



Machine learning: definition

- *A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E* [Mitchell]
- Problem definition for a learning agent
  - Task T
  - Performance measure P
  - Experience E

Designing a learning system

1. Choosing the training experience
   - Examples of best moves, games outcome …
2. Choosing the target function
   - board-move, board-value, …
3. Choosing a representation for the target function
   - linear function with weights (hypothesis space)
4. Choosing a learning algorithm for approximating the target function
   - A method for parameter estimation

Inductive learning

- Simplest form: learn a function from examples

f is the target function

An example is a pair (*x*, *f(x)*)

Problem: find a hypothesis *h*
such that $h \approx f$
given a training set of examples

(This is a highly simplified model of real learning:
- Ignores prior knowledge
- Assumes examples are given)

Model Selection: cross validation

```
function MODEL-SELECTION(Learner, examples, k) returns a (hypothesis, error rate) pair

  err ← an array, indexed by size, storing validation-set error rates
  training_set, test_set ← a partition of examples into two sets
  for size = 1 to ∞ do
      err[size] ← CROSS-VALIDATION(Learner, size, training_set, k)
      if err is starting to increase significantly then
          best_size ← the value of size with minimum err[size]
          h ← Learner(best_size, training_set)
          return h, ERROR-RATE(h, test_set)

function CROSS-VALIDATION(Learner, size, examples, k) returns error rate

  N ← the number of examples
  errs ← 0
  for i = 1 to k do
     validation_set ← examples[(i − 1) × N/k:i × N/k]
     training_set ← examples − validation_set
     h ← Learner(size, training_set)
     errs ← errs + ERROR-RATE(h, validation_set)
  return errs / k        // average error rate on validation sets, across k-fold cross-validation
```

Modeling Error during Cross Validation

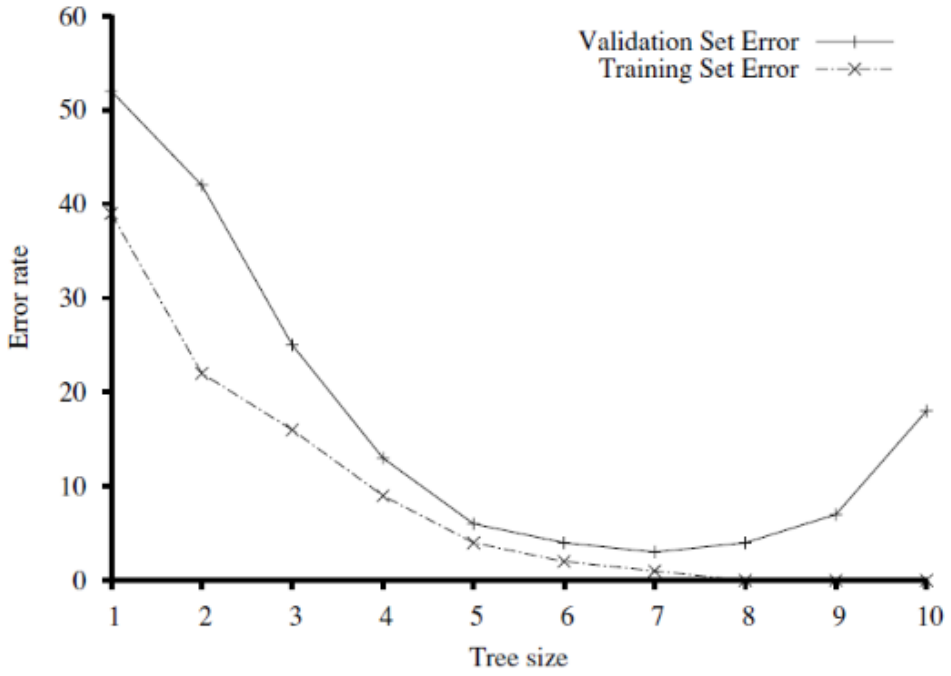


Figure 18.9 Error rates on training data (lower, dashed line) and validation data (upper, solid line) for different size decision trees. We stop when the training set error rate asymptotes, and then choose the tree with minimal error on the validation set; in this case the tree of size 7 nodes.

Model Selection: cross validation

```
function MODEL-SELECTION(Learner, examples, k) returns a (hypothesis, error rate) pair

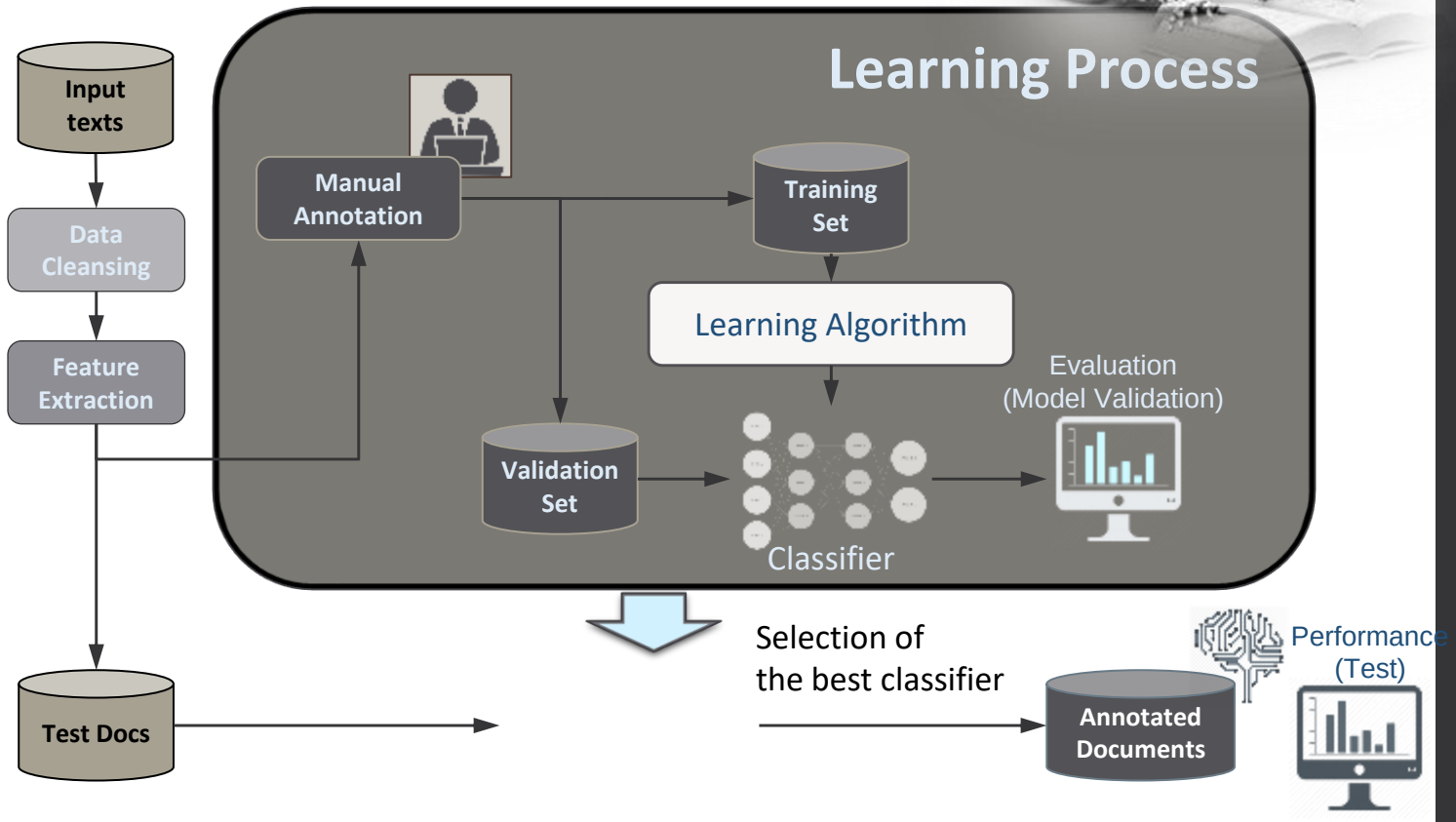
  err ← an array, indexed by size, storing validation-set error rates
  training_set, test_set ← a partition of examples into two sets
  for size = 1 to ∞ do
      err[size] ← CROSS-VALIDATION(Learner, size, training_set, k)
      if err is starting to increase significantly then
          best_size ← the value of size with minimum err[size]
          h ← Learner(best_size, training_set)
          return h, ERROR-RATE(h, test_set)

function CROSS-VALIDATION(Learner, size, examples, k) returns error rate

  N ← the number of examples
  errs ← 0
  for i = 1 to k do
    validation_set ← examples[(i − 1) × N/k:i × N/k]
    training_set ← examples − validation_set
    h ← Learner(size, training_set)
    errs ← errs + ERROR-RATE(h, validation_set)
  return errs / k        // average error rate on validation sets, across k-fold cross-validation
```

Figure 19.8 An algorithm to select the model that has the lowest validation error. It builds models of increasing complexity, and choosing the one with best empirical error rate, *err*, on the validation data set. *Learner*(*size*, *examples*) returns a hypothesis whose complexity is set by the parameter *size*, and which is trained on *examples*. In CROSS-VALIDATION, each iteration of the **for** loop selects a different slice of the *examples* as the validation set, and keeps the other examples as the training set. It then returns the average validation set error over all the folds. Once we have determined which value of the *size* parameter is best, MODEL-SELECTION returns the model (i.e., learner/hypothesis) of that size, trained on all the training examples, along with its error rate on the held-out test examples.

Machine Learning workflow



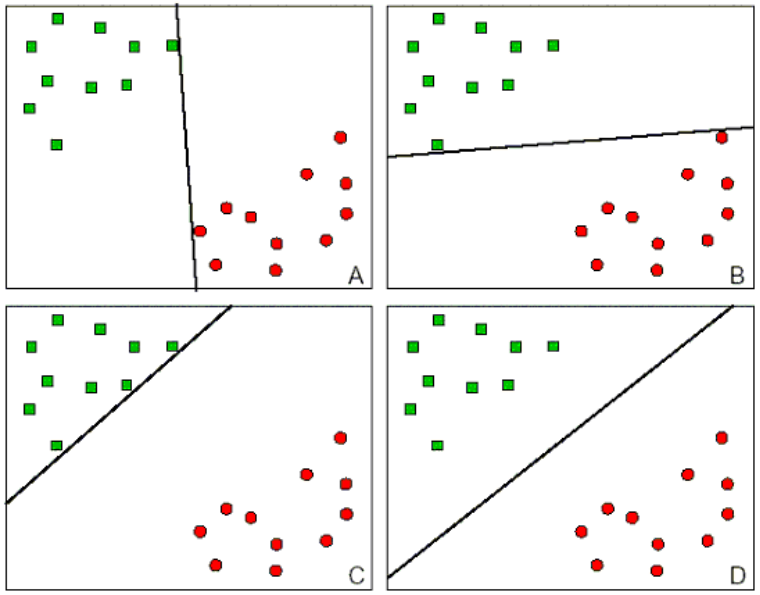
Metodi di Machine Learning

Tipi di Apprendimento

- Apprendimento Supervisionato
- Apprendimento Unsupervised
- Weakly Supervised Learning
- Reinforcement Learning

Metodi di ML: Classi di funzioni

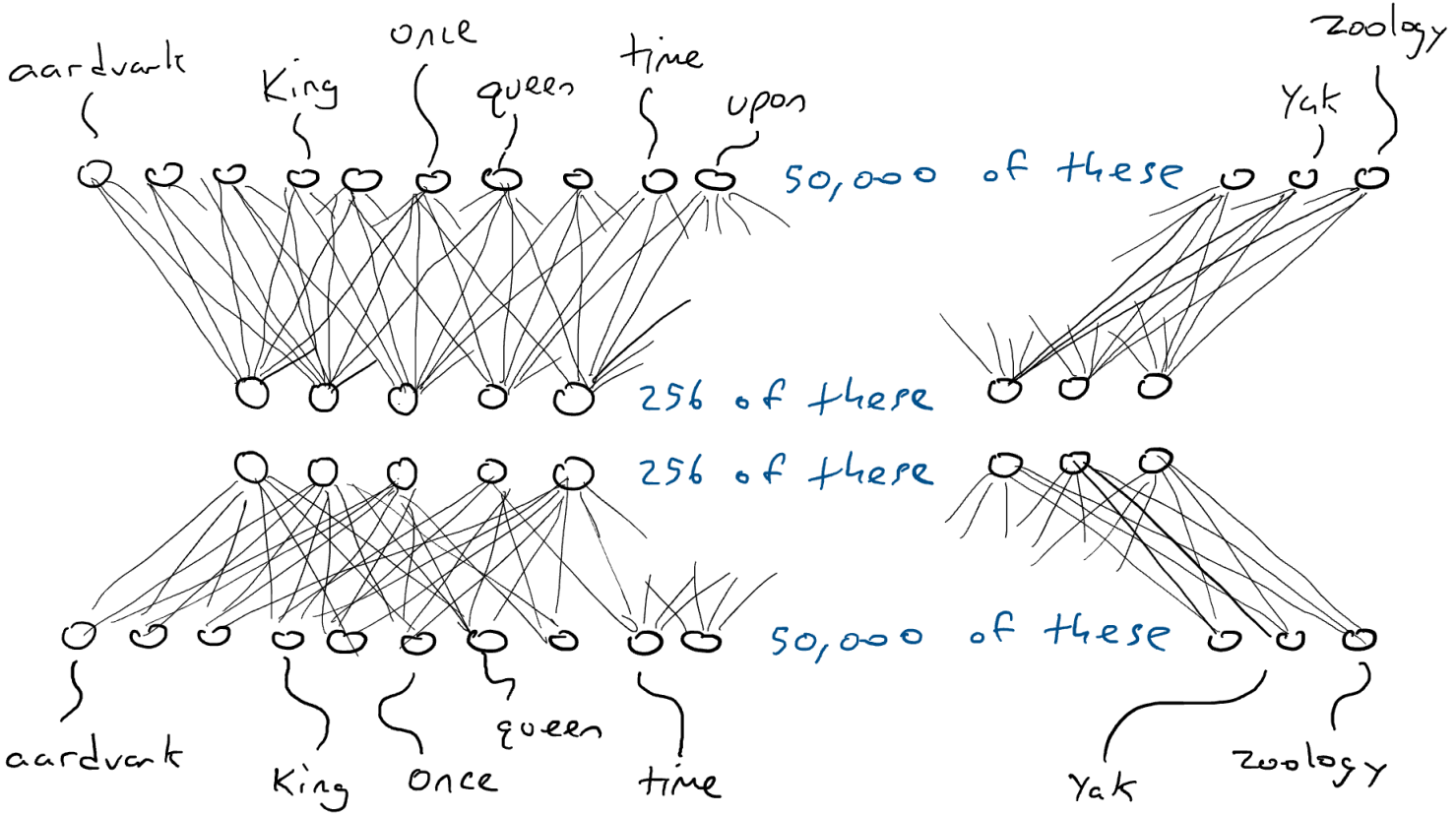
- **Approcci discriminativi**
  - Lineari
  - h(x) = sign(**W** · **x** + b)



  - Approcci probabilistici
    - Stima delle probabilità $p(\mathcal{C}_k|\mathbf{x})$ attraverso un training set
    - Modello generativo ed uso della inversione Bayesiana

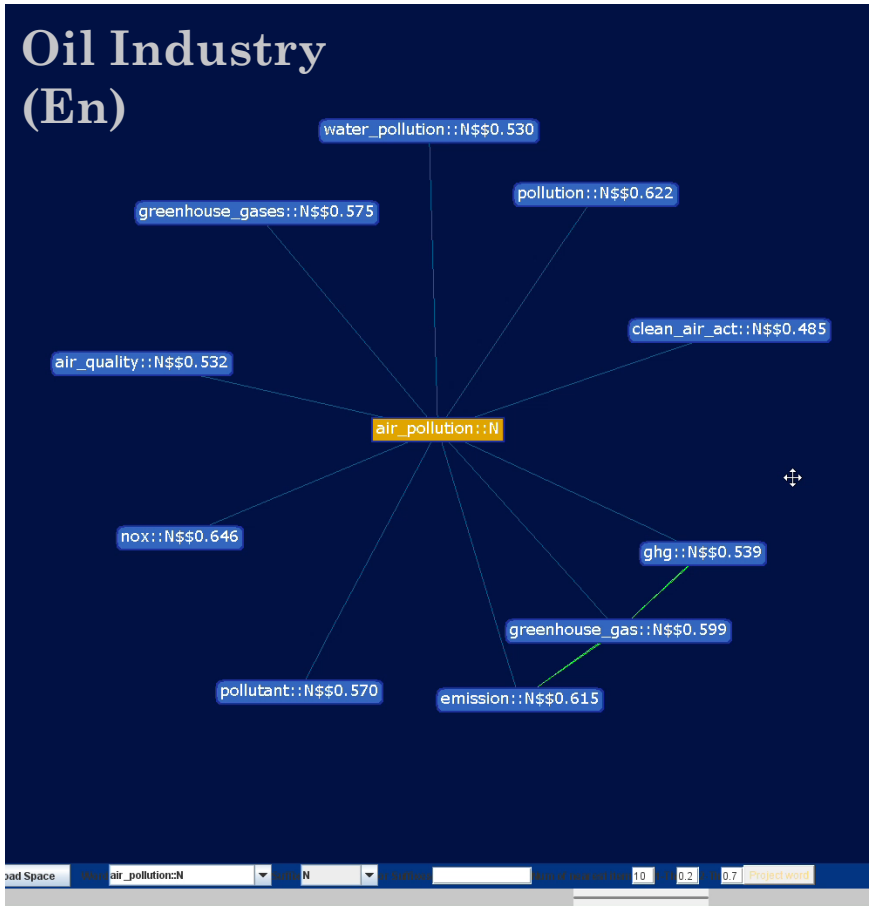
$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}.$$

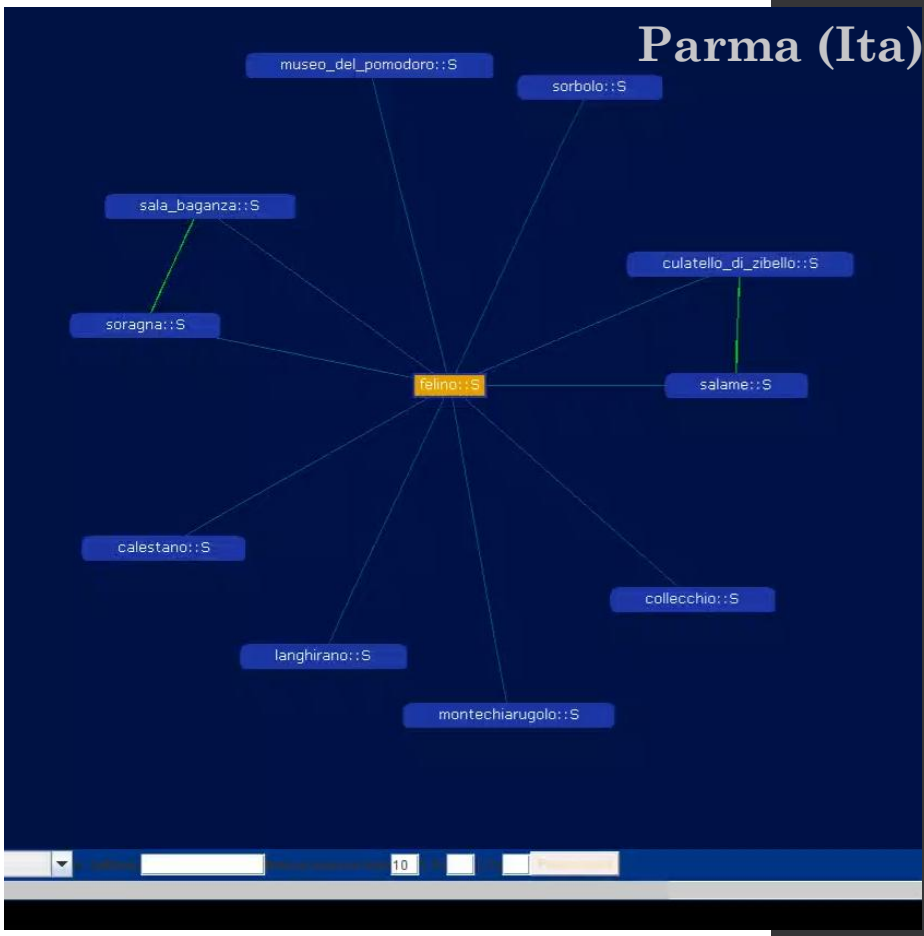
Linguistic Patterns and Machine Learning: Encoder-decoders for NL

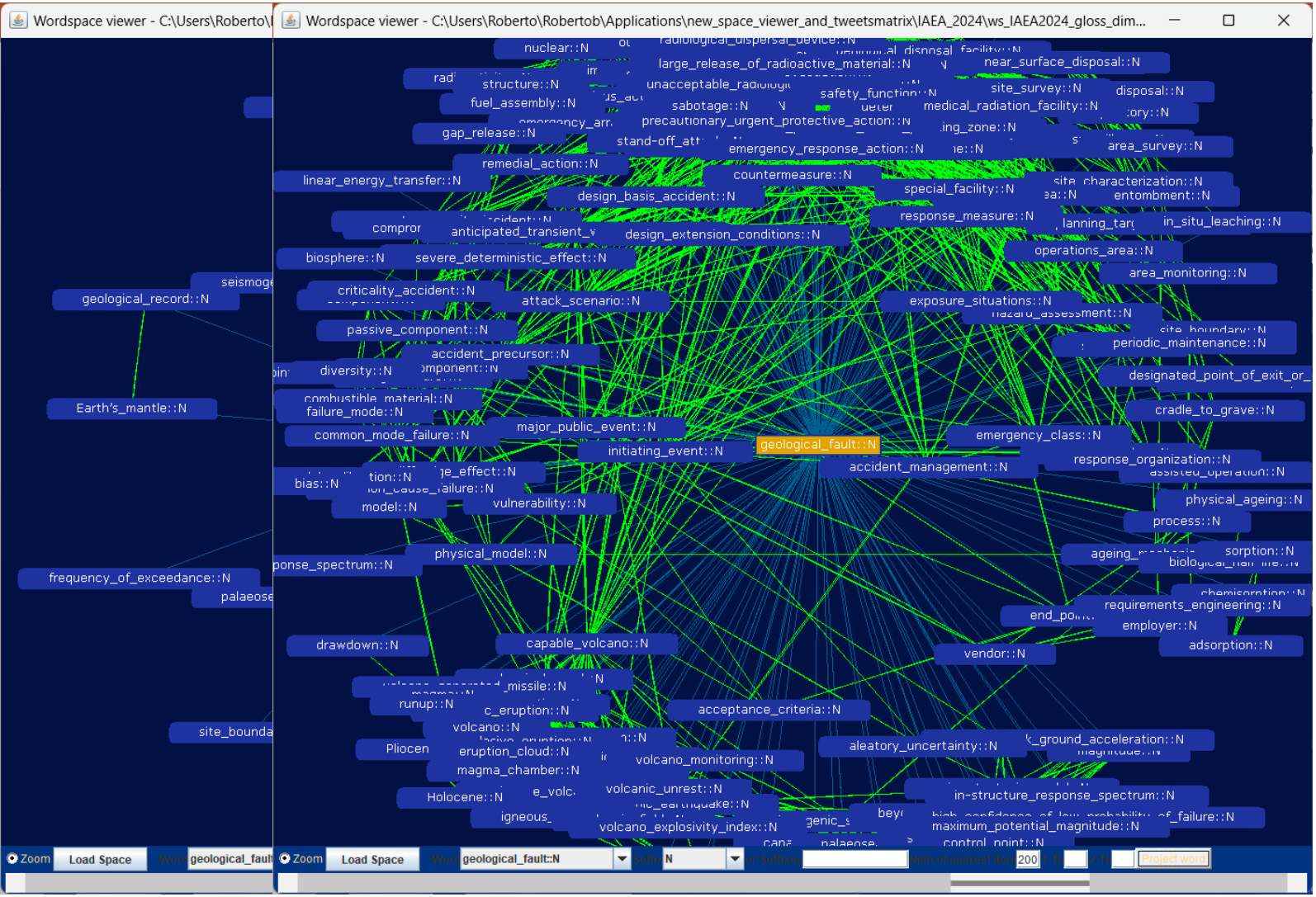


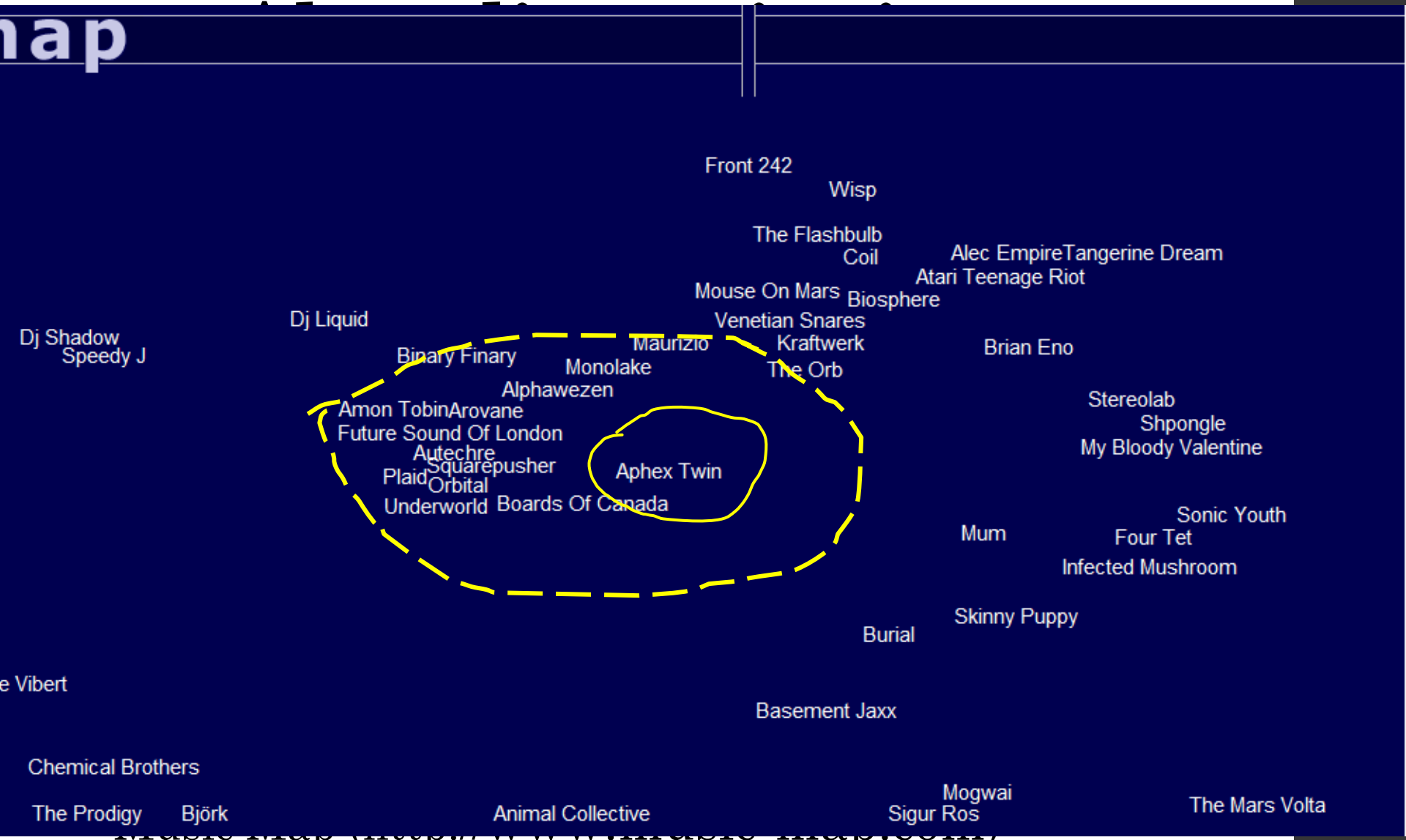
Semantic Embeddings from domain *Corpora*:

Wordspace (unsupervised induction of Vector representations)






Wordspace viewer - C:\Users\Roberto\Robertob\Applications\new_space_viewer_and_tweetsmatrix\IAEA_2024\ws_IAEA2024_gloss_dim...
nuclear::N
large_release_of_radioactive_material::N
near_surface_disposal::N
structure::N
fuel_assembly::N
sabotage::N
site_survey::N
disposal::N
medical_radiation_facility::N
precautionary_urgent_protective_action::N
gap_release::N
stand-off_att
emergency_response_action::N
area_survey::N
remedial_action::N
linear_energy_transfer::N
countermeasure::N
design_basis_accident::N
special_facility::N
entombment::N
response_measure::N
in_situ_leaching::N
anticipated_transient_
design_extension_conditions::N
biosphere::N
severe_deterministic_effect::N
operations_area::N
area_monitoring::N
criticality_accident::N
attack_scenario::N
exposure_situations::N
passive_component::N
periodic_maintenance::N
accident_precursor::N
diversity::N
designated_point_of_exit_or_
failure_mode::N
cradle_to_grave::N
major_public_event::N
common_mode_failure::N
initiating_event::N
geological_fault::N
emergency_class::N
accident_management::N
response_organization::N
bias::N
physical_ageing::N
model::N
vulnerability::N
process::N
physical_model::N
sorption::N
requirements_engineering::N
employer::N
drawdown::N
capable_volcano::N
vendor::N
adsorption::N
runup::N
acceptance_criteria::N
volcano::N
Pliocen
eruption_cloud::N
magma_chamber::N
volcano_monitoring::N
aleatory_uncertainty::N
Holocene::N
volcanic_unrest::N
in-structure_response_spectrum::N
igneous_
volcano_explosivity_index::N
maximum_potential_magnitude::N
control_point::N
geological_record::N
Earth's_mantle::N
frequency_of_exceedance::N
site_bounda
Zoom
Load Space
geological_fault::N
N
200
Project word


Front 242
Wisp
The Flashbulb
Coil
Alec Empire
Tangerine Dream
Atari Teenage Riot
Mouse On Mars
Biosphere
Dj Liquid
Venetian Snares
Dj Shadow
Speedy J
Maurizio
Kraftwerk
Brian Eno
Binary Finary
Monolake
The Orb
Alphawezen
Stereolab
Amon Tobin
Arovane
Shpongle
Future Sound Of London
My Bloody Valentine
Autechre
Squarepusher
Aphex Twin
Plaid
Orbital
Underworld
Boards Of Canada
Sonic Youth
Mum
Four Tet
Infected Mushroom
Skinny Puppy
Burial
e Vibert
Basement Jaxx
Chemical Brothers
Mogwai
The Prodigy
Björk
Animal Collective
Sigur Ros
The Mars Volta

Reti neurali

Breve Introduzione

Applicazioni

Modelli Avanzati

Linear Classification

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

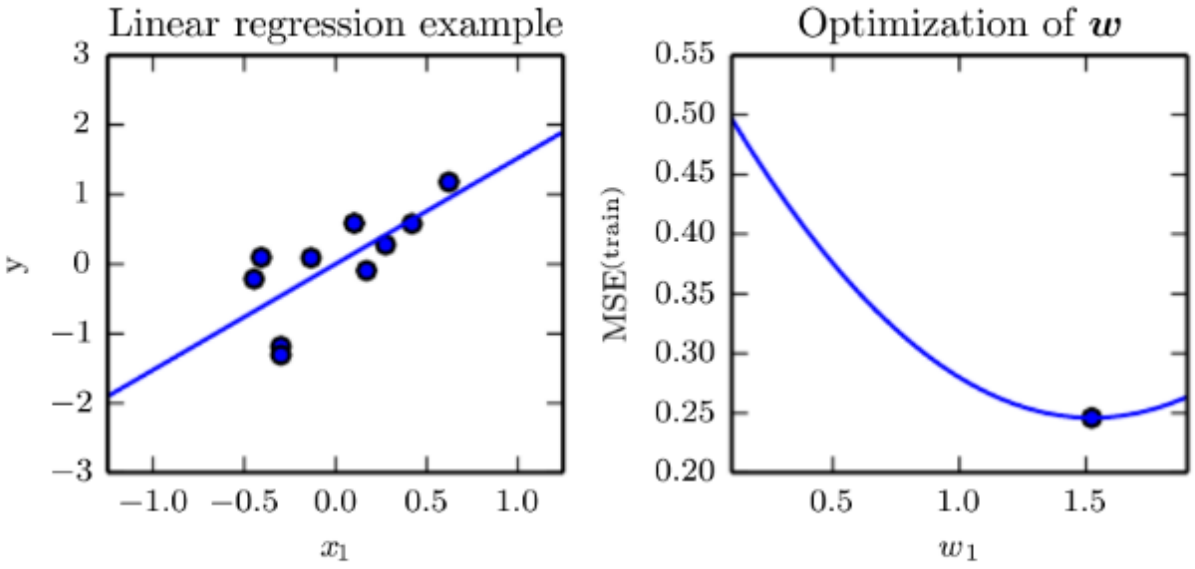


Figure 5.1: A linear regression problem, with a training set consisting of ten data points, each containing one feature. Because there is only one feature, the weight vector $\boldsymbol{w}$ contains only a single parameter to learn, w_1. *(Left)*Observe that linear regression learns to set w_1 such that the line $y = w_1 x$ comes as close as possible to passing through all the training points. *(Right)*The plotted point indicates the value of w_1 found by the normal equations, which we can see minimizes the mean squared error on the training set.

Linear Classification (2)

- Limitations:
  - Data **separability** (poor over non linear correlations)
  - **Hard (linearity) constraints** not suitable for **error-prone** data
  - Lack of expressiveness: **not so many parameters** to make proper use of several variables and their possible interactions

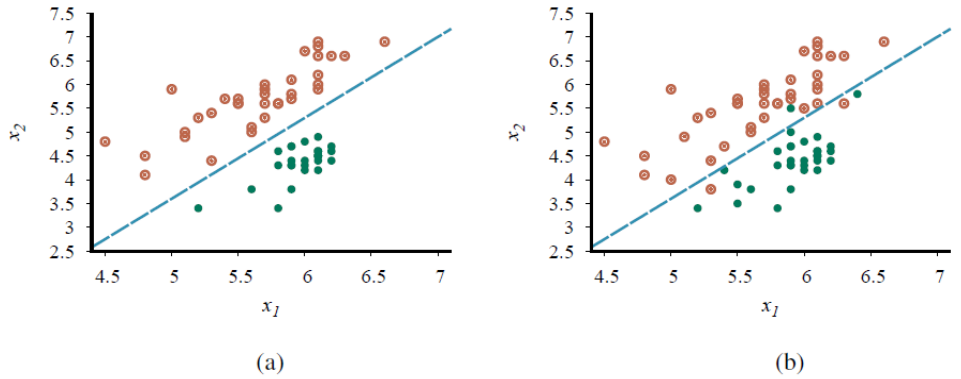
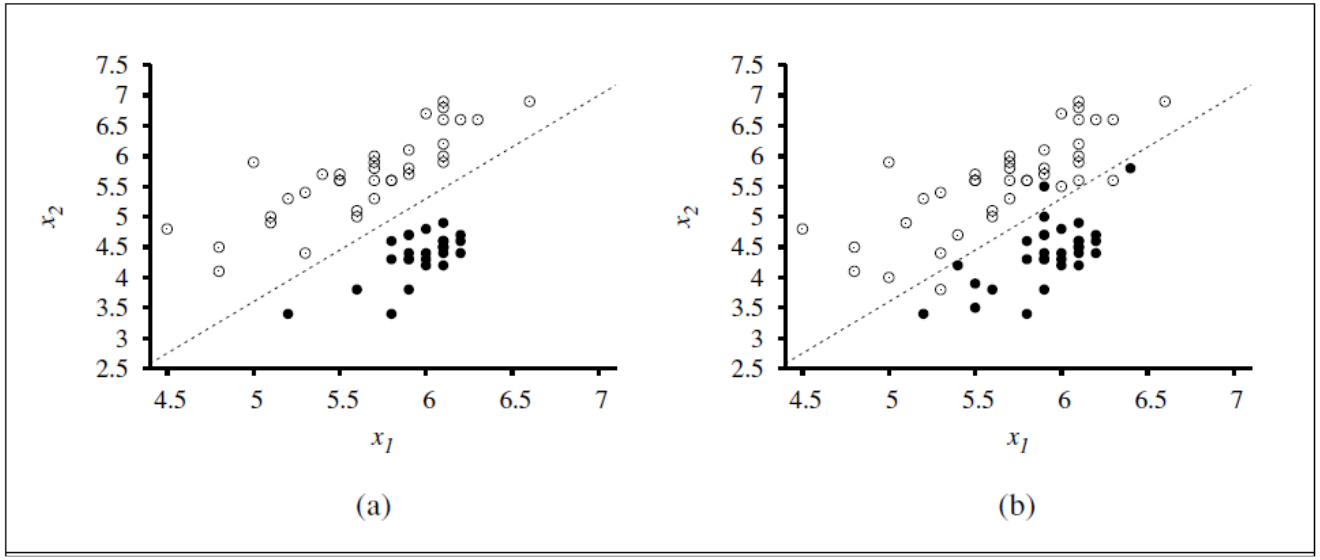


Figure 19.15 (a) Plot of two seismic data parameters, body wave magnitude x_1 and surface wave magnitude x_2, for earthquakes (open orange circles) and nuclear explosions (green circles) occurring between 1982 and 1990 in Asia and the Middle East (Kebeasy *et al.*, 1998). Also shown is a decision boundary between the classes. (b) The same domain with more data points. The earthquakes and explosions are no longer linearly separable.

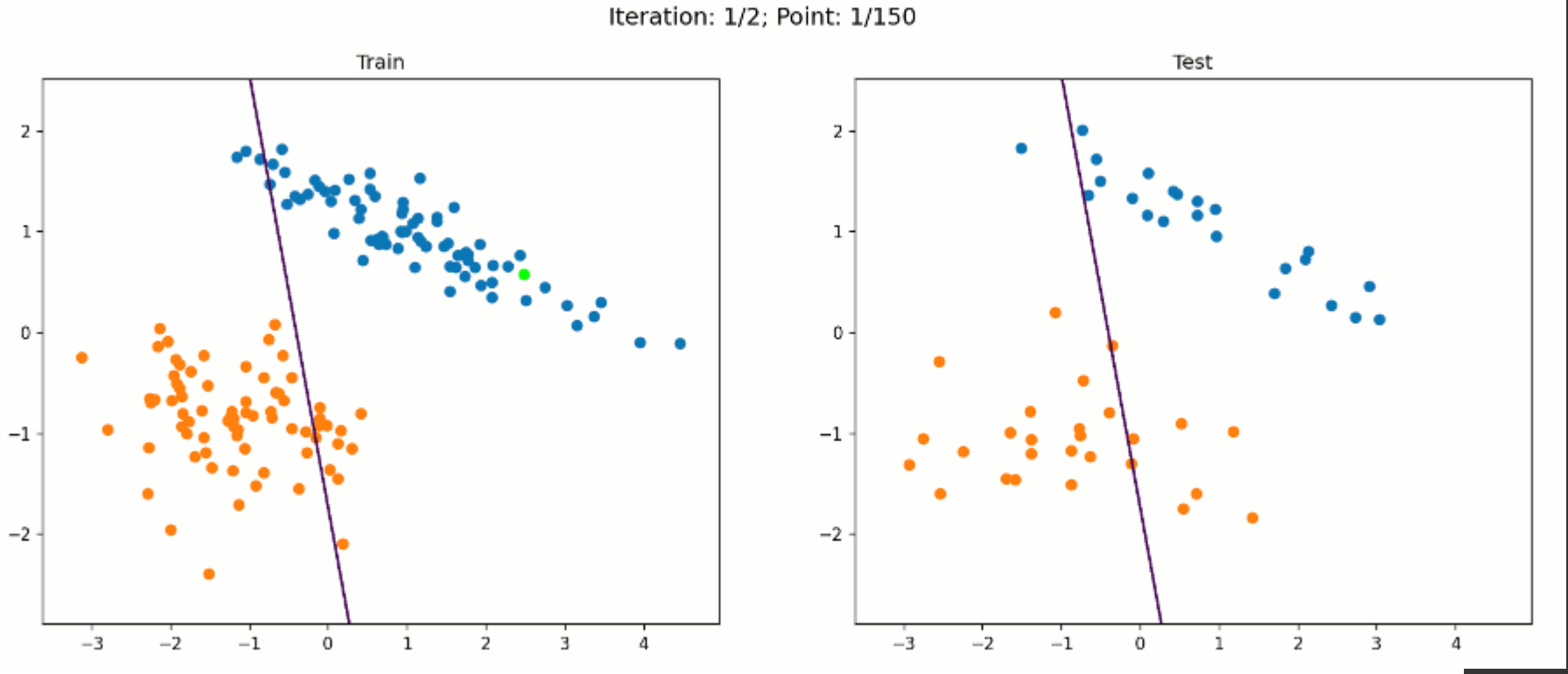
Linear Classification (3): an example



- Learned Model: $x_2 = 1.7x_1 - 4.9$ or $-4.9 + 1.7x_1 - x_2 = 0$.
- Separability Rule: $-4.9 + 1.7x_1 - x_2 > 0$

- Generalizing: $h_{\mathbf{w}}(\mathbf{x}) = 1$ if $\mathbf{w} \cdot \mathbf{x} \geq 0$ and 0 otherwise.
- How to achieve it?: $w_i \leftarrow w_i + \alpha\,(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$

… Adjusting weights



A reflection

- A linear classifier: $h_{\mathbf{w}}(x) = w_1 x + w_0$
- … with its error

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^{N} L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{i=1}^{N} (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{i=1}^{N} (y_j - (w_1 x_j + w_0))^2$$

- The best classifier is: $\mathbf{w}^* = \text{argmin}_{\mathbf{w}}\, Loss(h_{\mathbf{w}})$

that can be found via first derivatives of their arguments

$$\frac{\partial}{\partial w_0} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0$$

$$\frac{\partial}{\partial w_1} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0\,. \qquad (18.2)$$

- In general, for every i

$$\begin{aligned}\frac{\partial}{\partial w_i} Loss(\mathbf{w}) &= \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x))^2 \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - (w_1 x + w_0)) \,, \end{aligned} \tag{18.5}$$

- that results in:

$$\frac{\partial}{\partial w_0} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \,; \qquad \frac{\partial}{\partial w_1} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x$$

- with the general update rule such as:

$$w_0 \leftarrow w_0 + \alpha\,(y - h_{\mathbf{w}}(x)) \,; \quad w_1 \leftarrow w_1 + \alpha\,(y - h_{\mathbf{w}}(x)) \times x$$

N training point: batch GD

- Over N training points (j=1, …., N)

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j))$$

$$w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j$$

- Every set of N training instances is used to upgrade weights
- Upgrades are carried in a batch fashion with repeated scans of the entire training set

Learning Linear Classifier

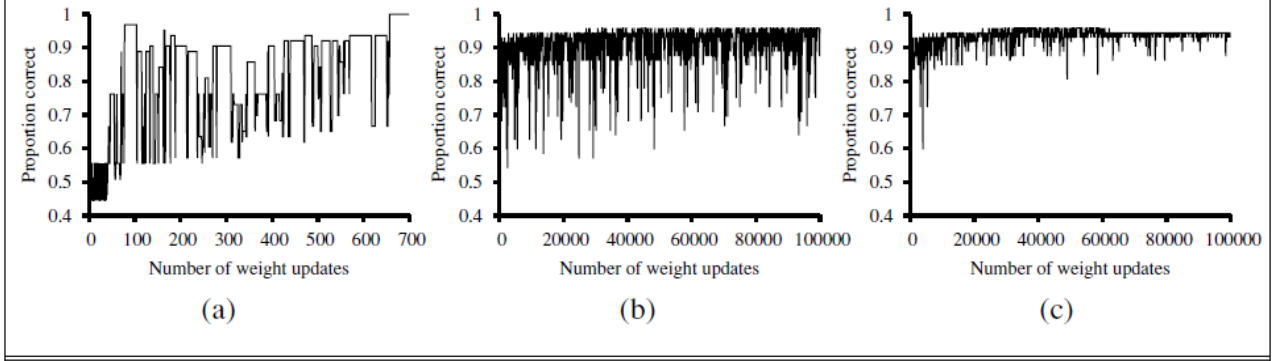


Figure 18.16 (a) Plot of total training-set accuracy vs. number of iterations through the training set for the perceptron learning rule, given the earthquake/explosion data in Figure 18.15(a). (b) The same plot for the noisy, non-separable data in Figure 18.15(b); note the change in scale of the x-axis. (c) The same plot as in (b), with a learning rate schedule $\alpha(t) = 1000/(1000 + t)$.

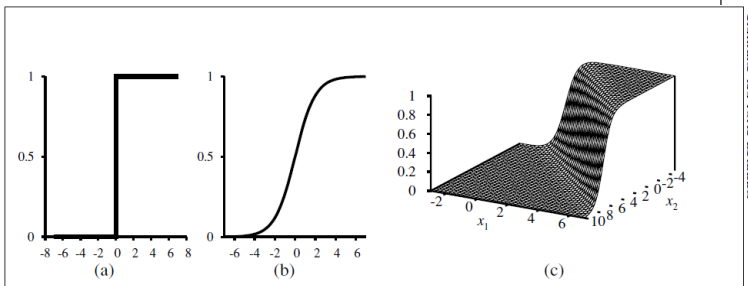


Figure 18.17 (a) The hard threshold function $Threshold(z)$ with 0/1 output. Note that the function is nondifferentiable at $z = 0$. (b) The logistic function, $Logistic(z) = \frac{1}{1+e^{-z}}$, also known as the sigmoid function. (c) Plot of a logistic regression hypothesis $h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x})$ for the data shown in Figure 18.15(b).

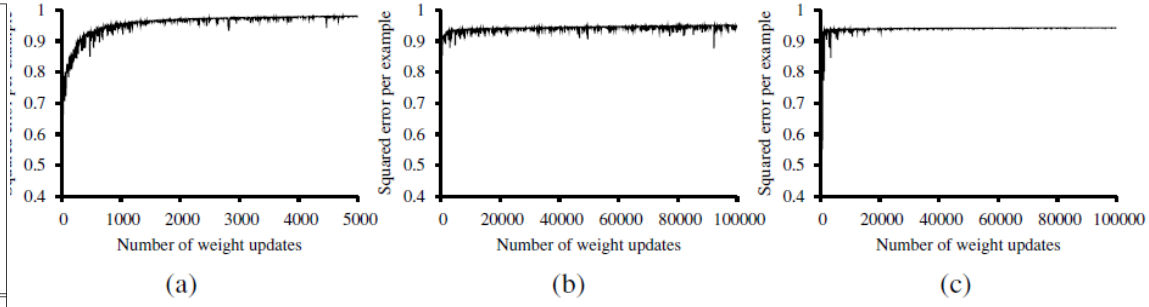
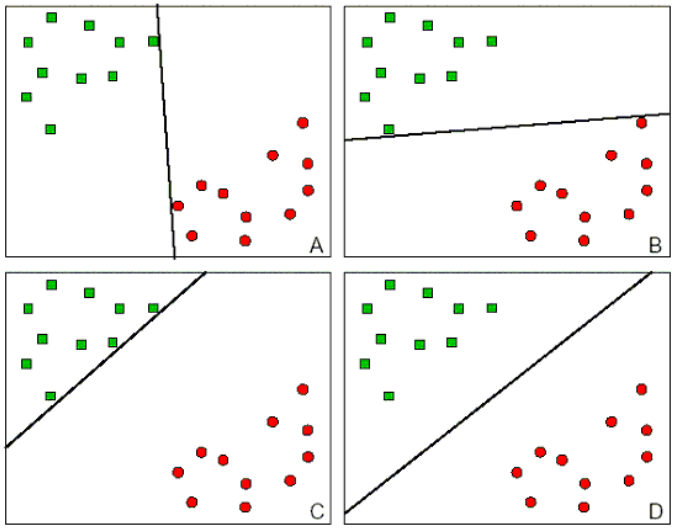
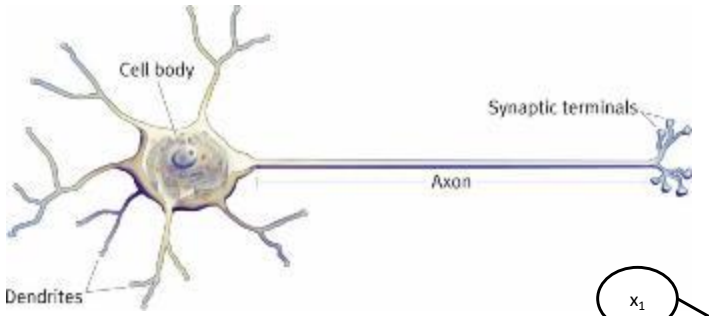


Figure 18.18 Repeat of the experiments in Figure 18.16 using logistic regression and squared error. The plot in (a) covers 5000 iterations rather than 1000, while (b) and (c) use the same scale.

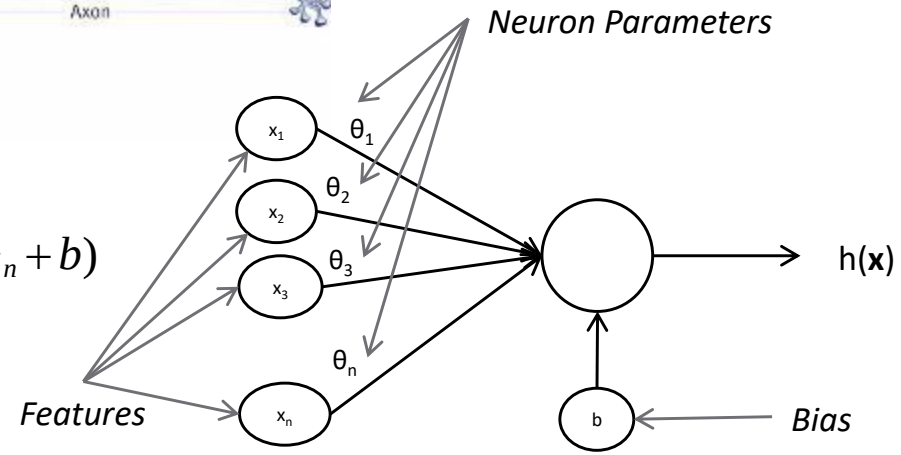
Perceptron (Rosenblatt, 1958)

- Linear Classifier mimicking a neuron

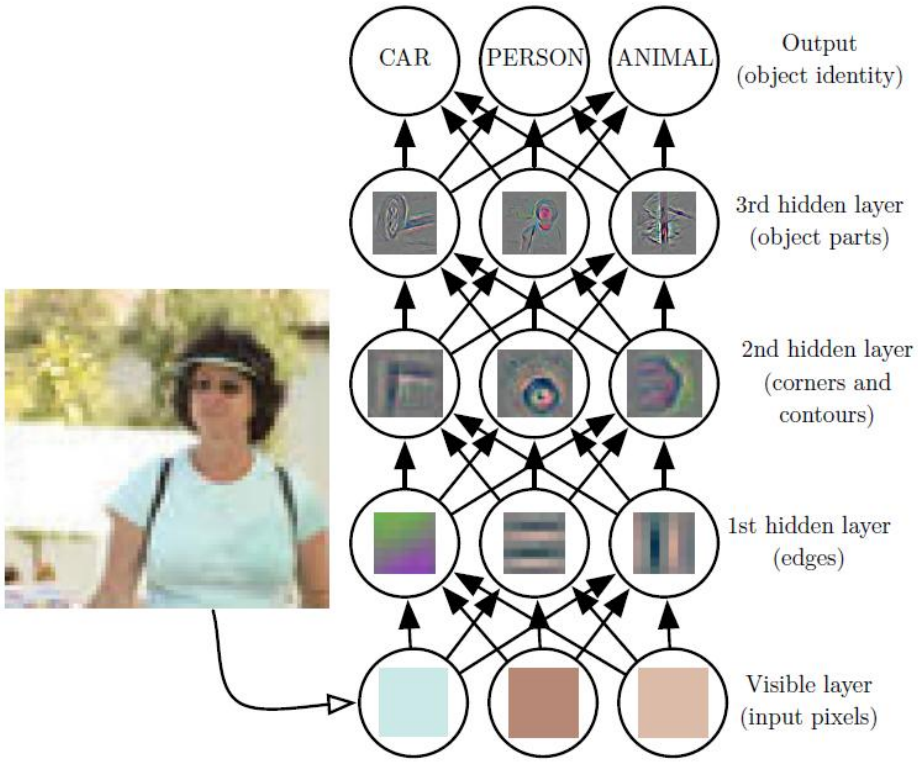




$$h(\vec{x}) = g(\sum_n \theta_n x_n + b)$$



Representation and Learning



Zeiler and Fergus (2014)

Networks and Information Flow

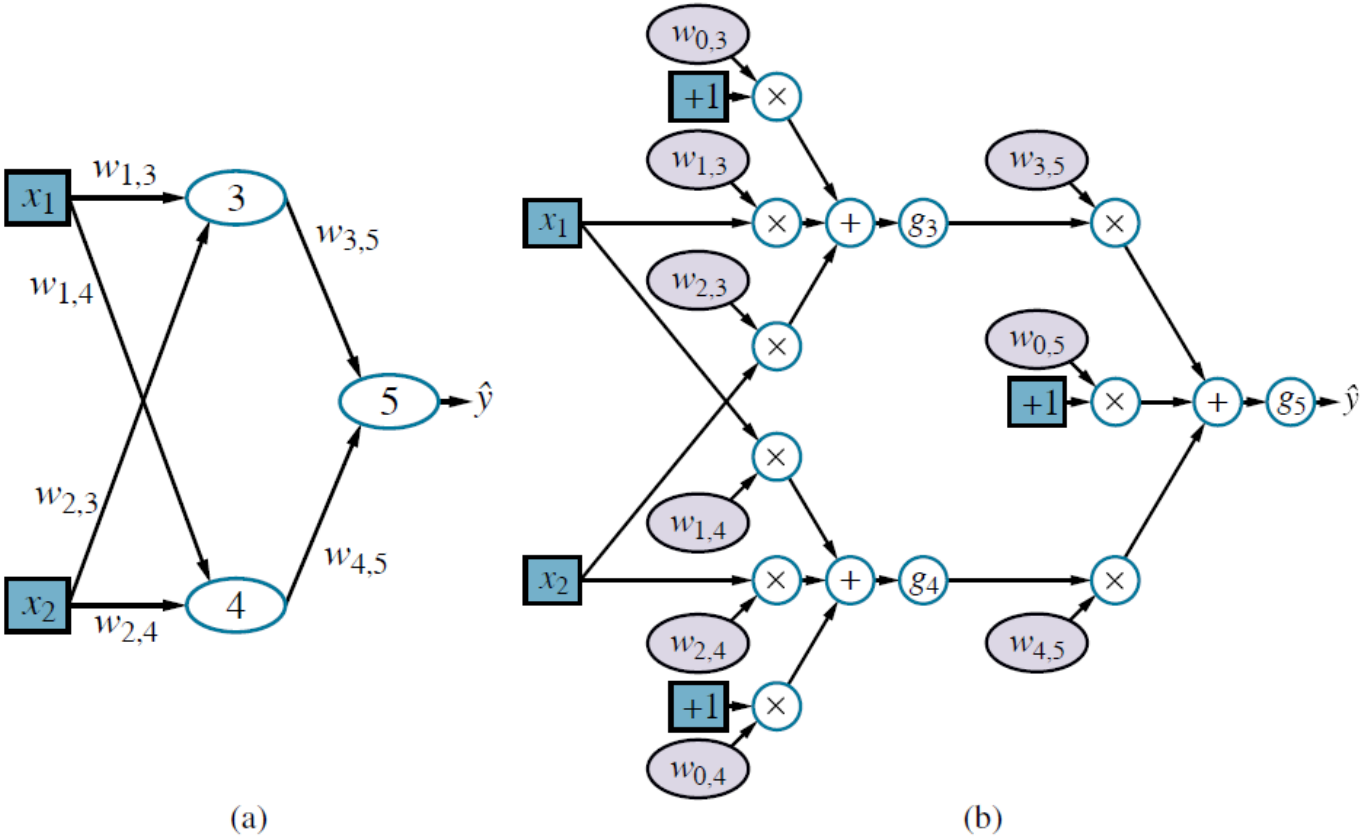
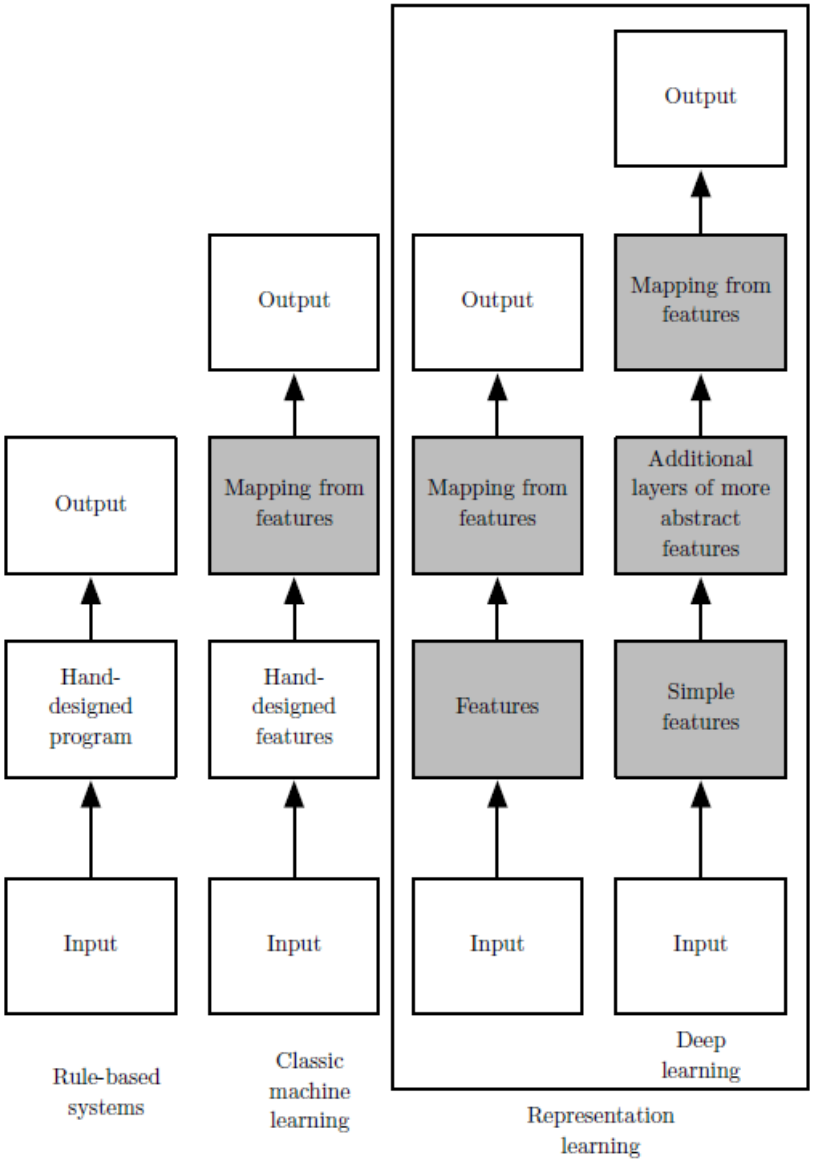


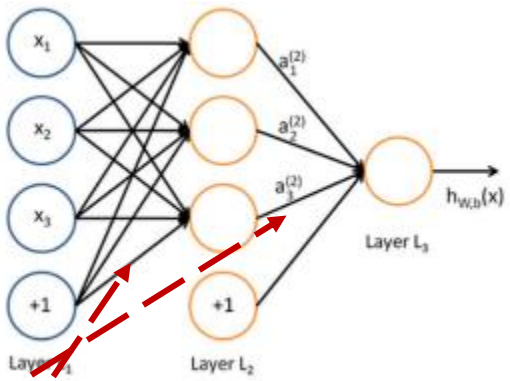
Figure 22.3 (a) A neural network with two inputs, one hidden layer of two units, and one output unit. Not shown are the dummy inputs and their associated weights. (b) The network in (a) unpacked into its full computation graph.

Learning Multiple Components



from Goodfellow et al., DL MIT book

Adding Layers …



- From simple linear laws …

$$h(\vec{x}) = g(\vec{x};\vec{\theta},b) = g(\sum_n \theta_n x_n + b)$$

- to feedforward structures. It can be made dependent on a sequence of functions $g^{(1)}$ and $g^{(2)}$, …, $g^{(k)}$ that give rise to a structured hypothesis:

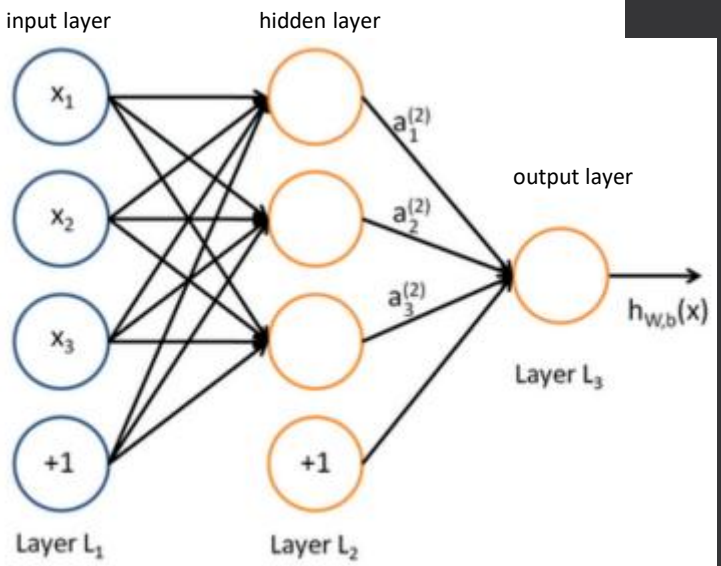
$$h(\vec{x}) = g^{(2)}(g^{(1)}(\vec{x};\vec{\theta}^{(1)},b^{(1)});\vec{\theta}^{(2)},b^{(2)}) =$$
$$= W^{(2)}g^{(2)}(g^{(1)}(W^{(1)}\cdot\vec{x}+b^{(1)})+b^{(2)}$$

- Hidden layer

$$h^{(1)}(\vec{x}) = g^{(1)}(W^{(1)}\vec{x}+b^{(1)})$$

Artificial Neural Networks

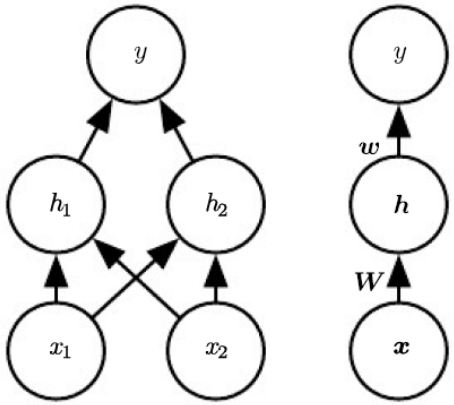
- Each circle represent a **neuron** (or unit)
  - In Figure: 3 **input**, 3 **hidden** and 1 **output**
- n_l=3 is the **number of layers**
- s_l denotes the **number of units in layer *l***
- Layers:
  - Layer l is denoted as L_l
  - Layer l and $l+1$ are connected by a matrix $W^{(l)}$ of parameters
    - $W^{(l)}_{i,j}$ connects neuron j in layer l with neuron i in layer $l+1$
- $b^{(l)}_i$ is the *bias* associated to neuron i in layer $l+1$



$$h(\vec{x}) = g^{(2)}(g^{(1)}(\vec{x};\vec{\theta}^{(1)},b^{(1)});\vec{\theta}^{(2)},b^{(2)}) =$$
$$= W^{(2)}g^{(2)}(g^{(1)}(W^{(1)}\cdot\vec{x}+b^{(1)})+b^{(2)}$$

$$h^{(1)}(\vec{x}) = g^{(1)}(W^{(1)}\vec{x}+b^{(1)})$$

Training MLPs: Back-propagation



- How are parameters of the two-layer network, i.e. W, w and c, b defined?
- This is the role of the training algorithm for which:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^\top \max\{0, \boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{c}\} + b \approx f^*(\boldsymbol{x}).$$

- The learning process in MLPs is based on two notions:
  - The optimization local to individual neurons
  - The adjustments to the overall network by propagation backwards from the output (where the error manifests) through all the hidden layers.

Classification and Learning

- Forward classification vs.
  backpropagation learning

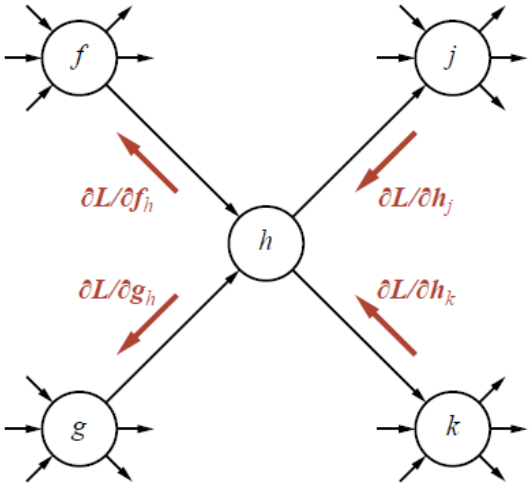
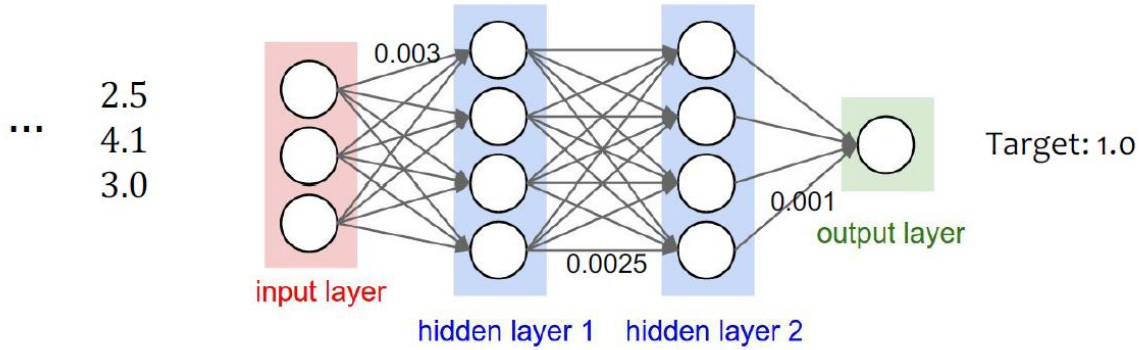
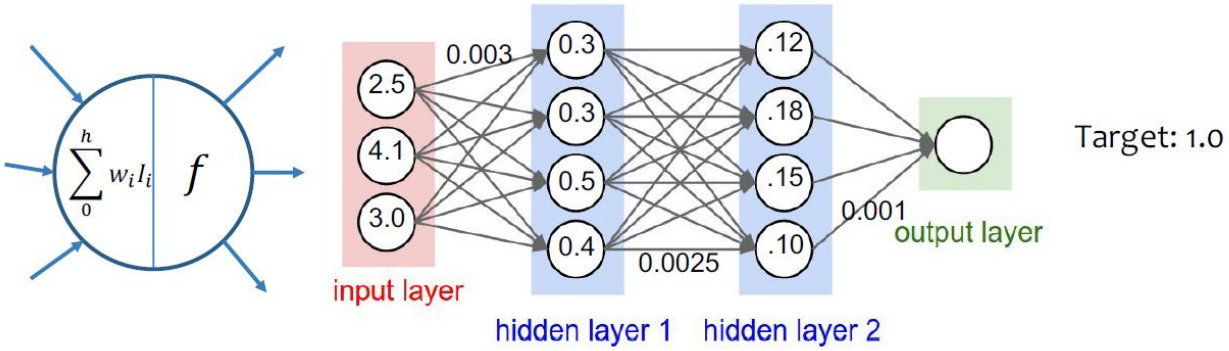


Figure 22.6 Illustration of the back-propagation of gradient information in an arbitrary computation graph. The forward computation of the output of the network proceeds from left to right, while the back-propagation of gradients proceeds from right to left.

Forward Steps

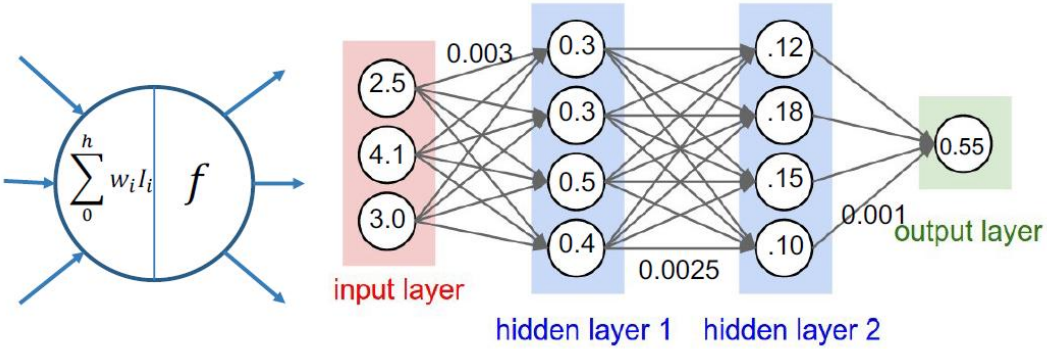


Weight Initialization and Loading the Data



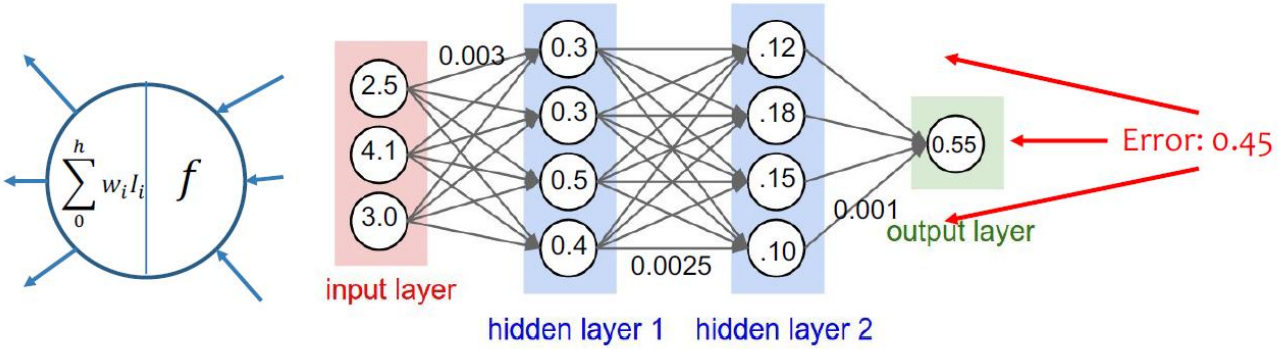
Going forward (Regularization, Dropout, Batch Normalization, …)

… and Backward steps



Target: 1.0

$Loss\ Function: (T - O) \Rightarrow Loss: 0.45$



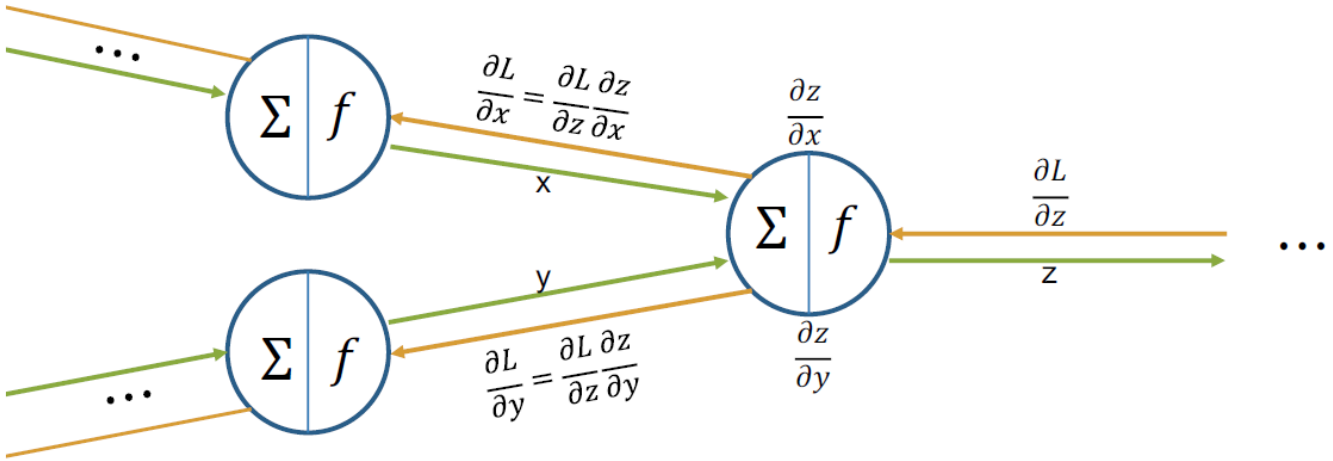
Updating the weights using Backpropagation $\frac{\partial E}{\partial w_j^i}$

How to induce the hypothesis *h* from examples

- Learn the parameters θ and b
- To find these we look at the past data (i.e. training data) optimizing an objective function
- Objective function: the error we make on the training data
  - the sum of differences between the decision function h and the label y
  - also called Loss Function or Cost Function

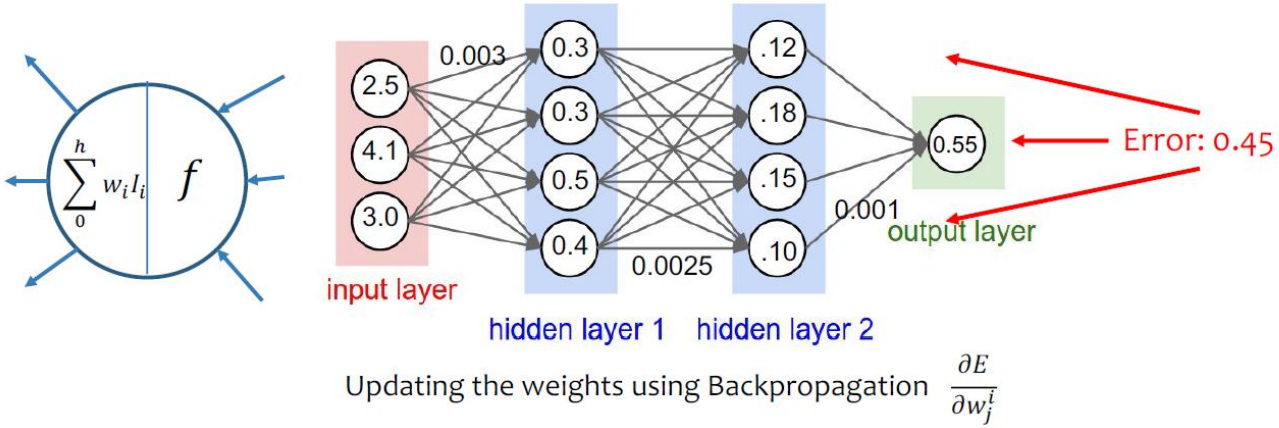
$$J(\theta, b) = \sum_{i=1}^{m} (h(x^{(i)}; \theta, b) - y^{(i)})^2$$

BP as Local Search

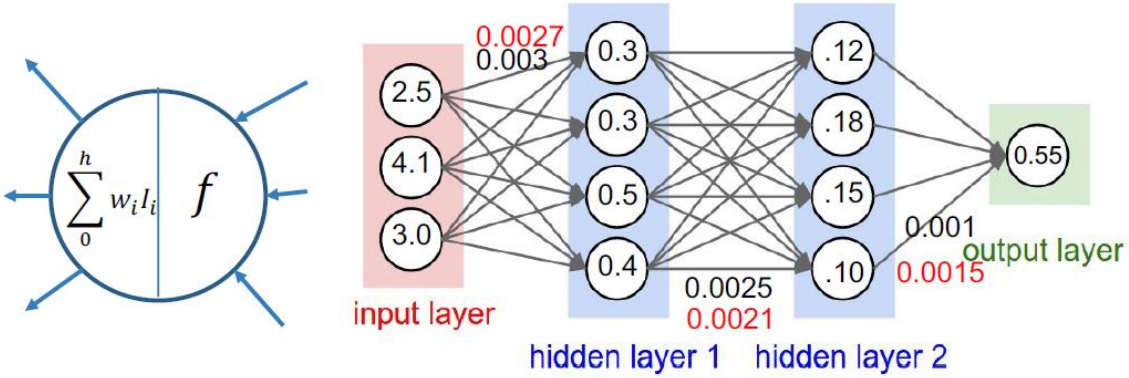


Backpropagation

Reweighting through GD



Updating the weights using Backpropagation $\frac{\partial E}{\partial w_j^i}$



By Choosing the Optimizer, new weights will be computed $w_{new} = w - \eta \frac{\partial E}{\partial w}$

A general training procedure: Stochastic Gradient Descent

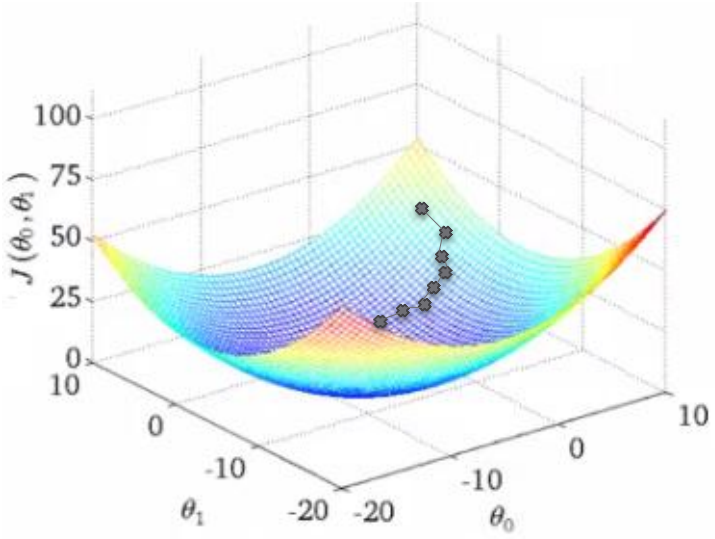
- Optimizing J means minimizing it
  - it measures the errors we make on the training data.
- We can iterate over examples and update the parameters in the direction of smaller costs
  - we aim at finding the minimum of that function
- Concretely,

$$\theta_1 = \theta_1 - \alpha \Delta \theta_1$$
$$\theta_2 = \theta_2 - \alpha \Delta \theta_2$$
$$b = b - \alpha \Delta b$$

- α is a meta-parameter, the learning rate
- Δ are the partial derivatives of the cost function wrt each parameter

Why SGD?

- Weights are updated using the partial derivatives
- Derivative pushes down the cost following the steepest descent path on the error curve



SGD procedure

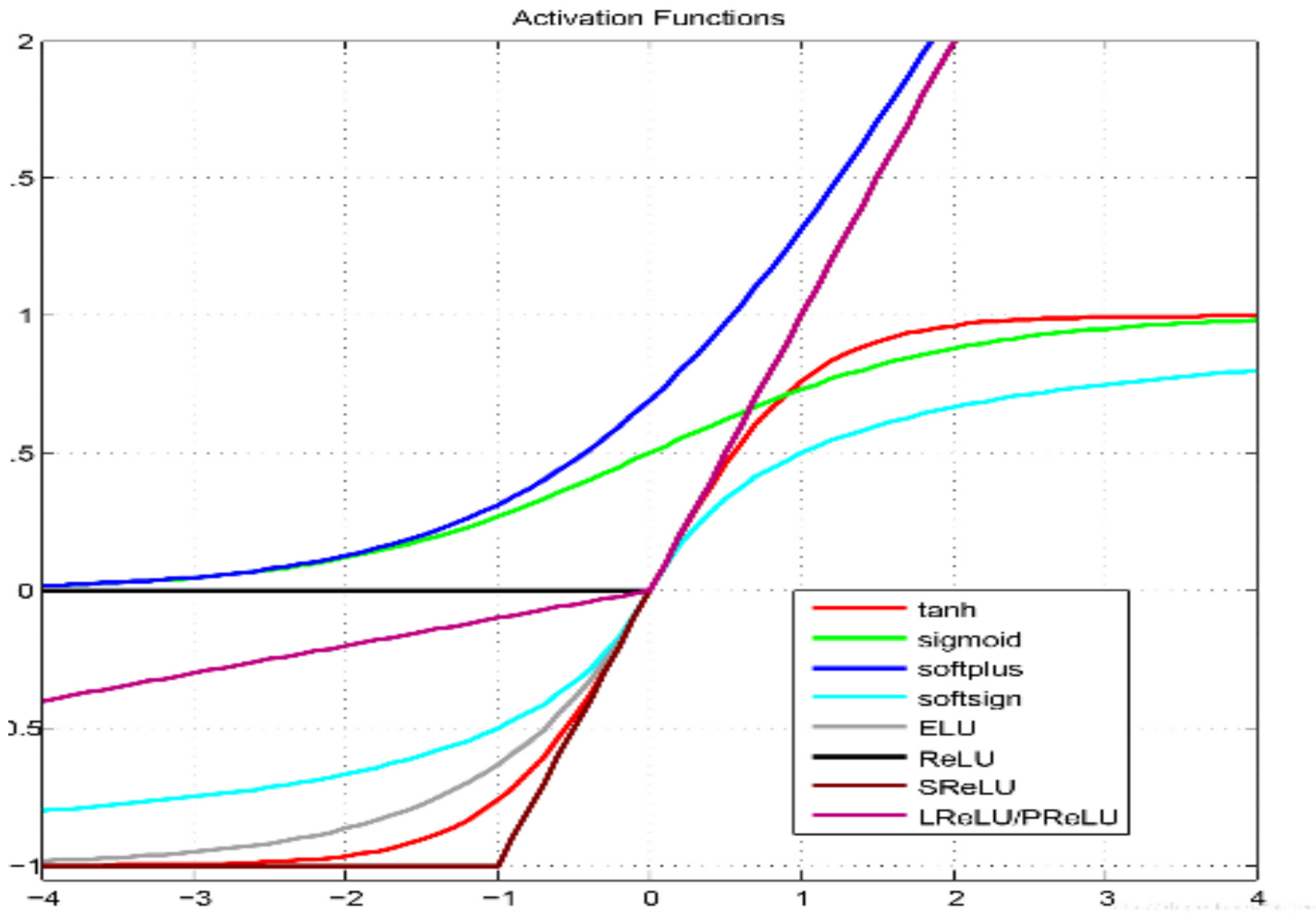
- Choose an initial random values for θ and b
- Choose a learning rate
- Repeat until stop criterion is met:
  - Pick a random training example x(i)
  - Update the parameters with

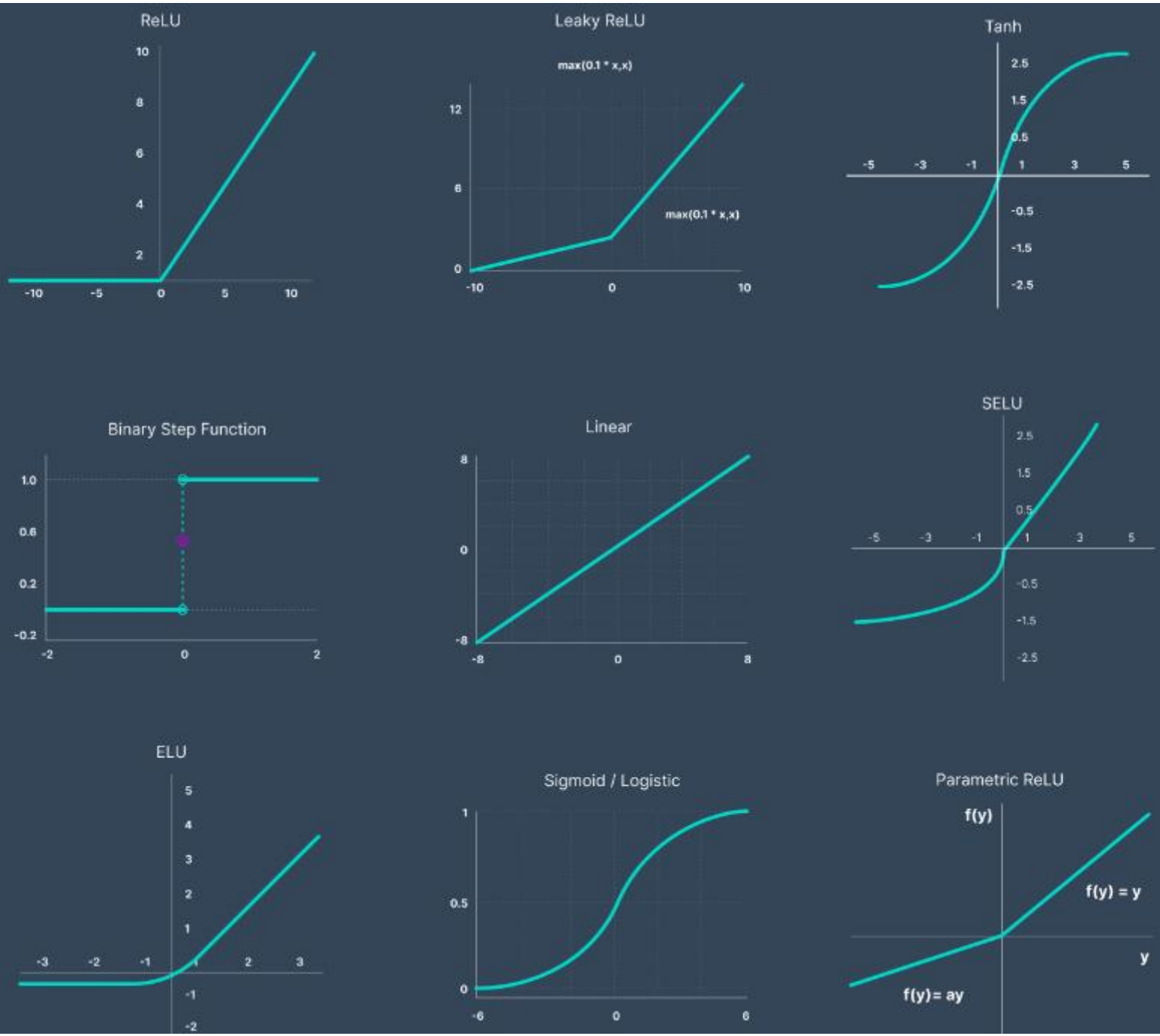
$$\theta_1 = \theta_1 - \alpha \Delta \theta_1$$
$$\theta_2 = \theta_2 - \alpha \Delta \theta_2$$
$$b = b - \alpha \Delta b$$

- We can stop
  - when the parameters do not change or,
  - the number of iteration exceeds a certain upper bound

… a bit of math …

Perceptron and non-linear activation functions




ReLU
10
8
6
4
2
-10
-5
0
5
10
Leaky ReLU
max(0.1 * x,x)
12
6
0
-10
0
10
max(0.1 * x,x)
Tanh
2.5
1.5
0.5
-5
-3
-1
1
3
5
-0.5
-1.5
-2.5
Binary Step Function
1.0
0.6
0.2
-0.2
-2
0
2
Linear
8
0
-8
-8
0
8
SELU
2.5
1.5
0.5
-5
-3
-1
1
3
5
-0.5
-1.5
-2.5
ELU
5
4
3
2
1
-3
-2
-1
1
2
3
-1
-2
Sigmoid / Logistic
1
0.5
0
-6
0
6
Parametric ReLU
f(y)
f(y) = y
y
f(y)= ay

How to induce h from examples

- Learn the parameters θ and b
- To find these we look at the past data (i.e. training data) optimizing an objective function
- **Objective function**: the error we make on the training data
  - the sum of differences between the decision function h and the label y
  - also called **Loss Function** or **Cost Function**

$$J(\theta, b) = \sum_{i=1}^{m} (h(x^{(i)}; \theta, b) - y^{(i)})^2$$

A general training procedure: Stochastic Gradient Descent

- Optimizing J means **minimizing** it
  - it measures the errors we make on the training data.
- We can iterate over examples and update the parameters in the direction of smaller costs
  - we aim at finding the minimum of that function
- Concretely,

$$\theta_1 = \theta_1 - \alpha\Delta\theta_1$$
$$\theta_2 = \theta_2 - \alpha\Delta\theta_2$$
$$b = b - \alpha\Delta b$$

- α is a meta-parameter, the learning rate
- Δ are the partial derivatives of the cost function *wrt* each parameter

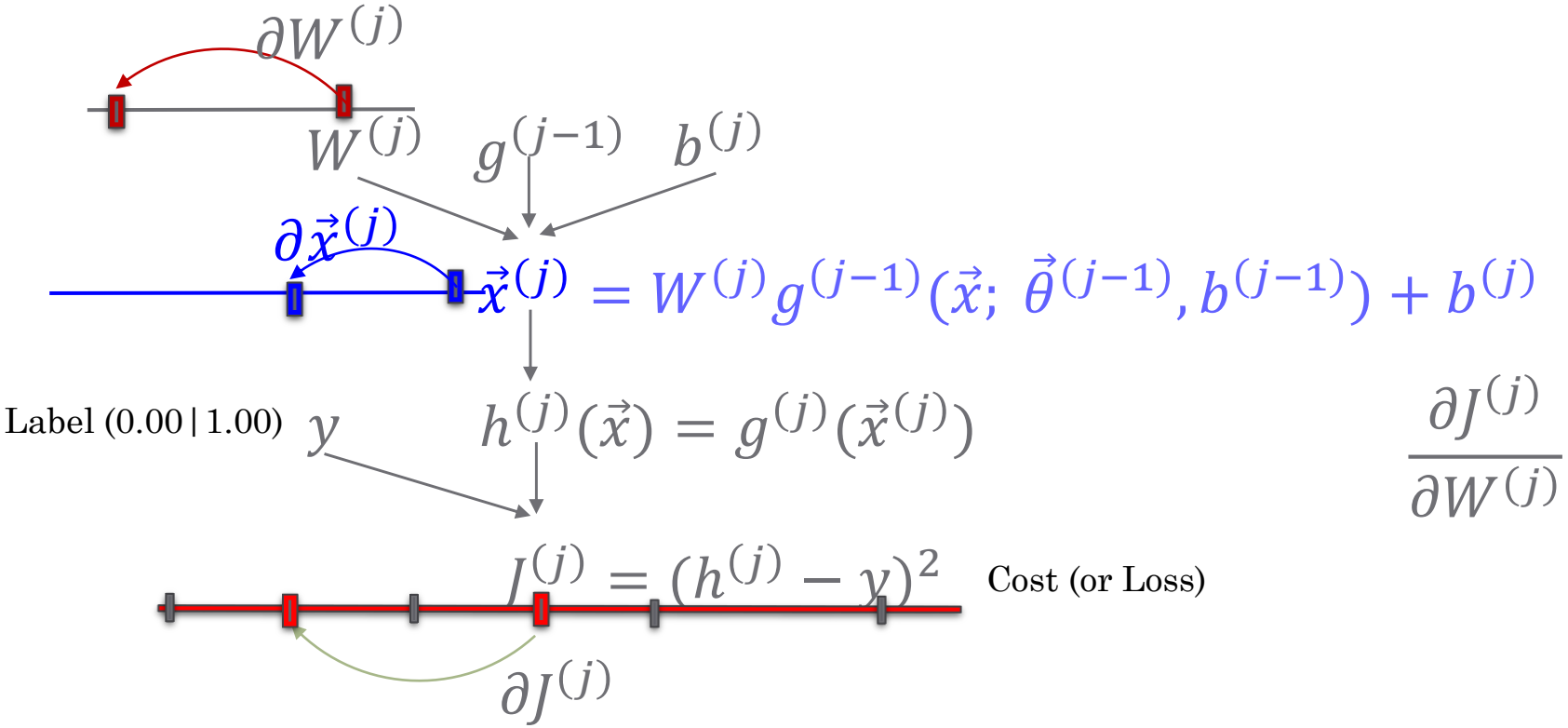
Optimizing J

- From the network

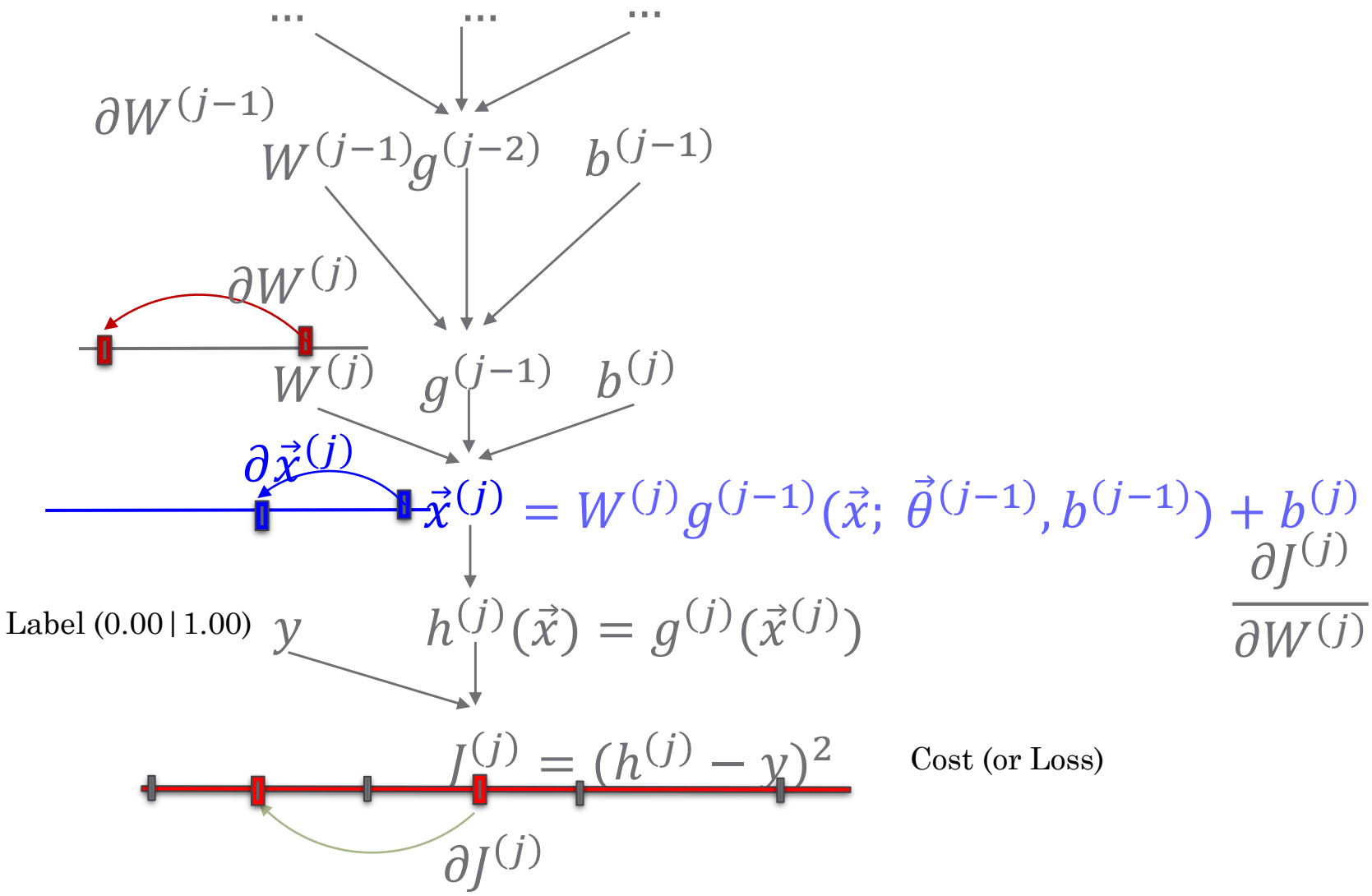
$$h(\vec{x}) = g^{(k)}\left(g^{(k-1)}(\ldots g^{(1)}(\vec{x};\vec{\theta}^{(1)},b^{(1)});\ldots);\vec{\theta}^{(k-1)},b^{(k-1)});\vec{\theta}^{(k)},b^{(k)}\right)=$$
$$= g^{(k)}\left(W^{(k)}g^{(k-1)}(W^{(k-1)}\ldots.g^{(1)}(W^{(1)}\vec{x}+\ b^{(1)})\ldots+b^{(k-1)})+b^{(k)}\right)$$

- and j-th layers equation:

$$h^{(j)}(\vec{x}) = g^{(j)}\left(W^{(j)}g^{(j-1)}(\vec{x};\ \vec{\theta}^{(j-1)},\ \ b^{(j-1)})+b^{(j)}\right) \qquad j=2,\ldots.,k-1$$

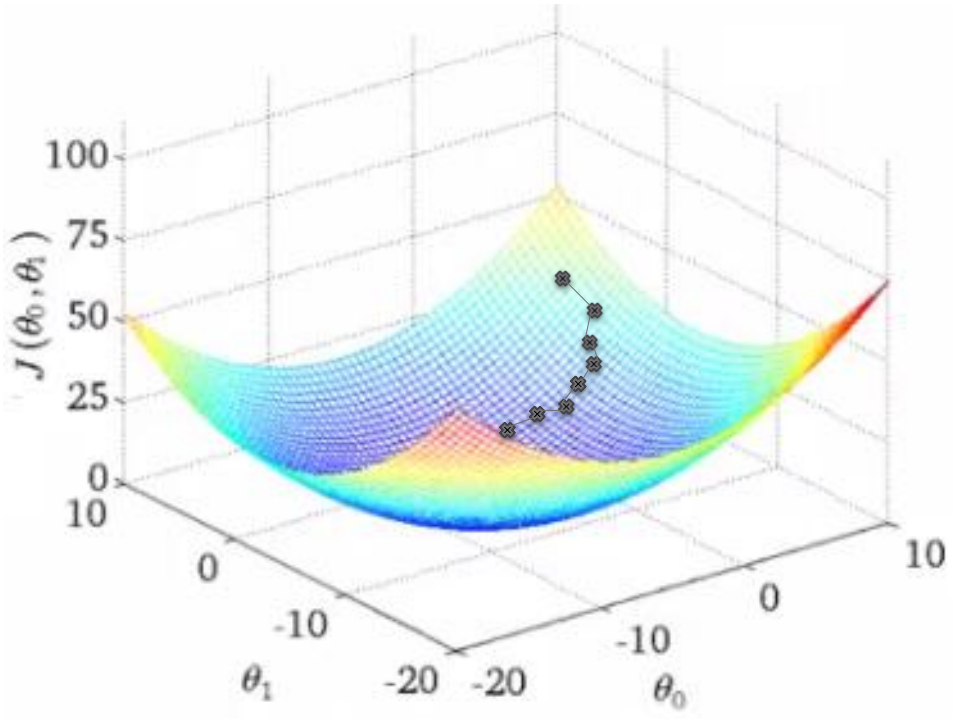


Optimizing J … backwards



Why SGD?

- Weights are updated using the partial derivatives
- Derivative pushes down the cost following the steepest descent path on the error curve



SGD procedure

- Choose an initial random values for θ and b
- Choose a learning rate
- Repeat until stop criterion is met:
  - Pick a random training example $x^{(i)}$
  - Update the parameters with

$$\theta_1 = \theta_1 - \alpha\Delta\theta_1$$
$$\theta_2 = \theta_2 - \alpha\Delta\theta_2$$
$$b = b - \alpha\Delta b$$

- We can stop **WHEN**
  - when the **parameters do not change** or,
  - the **number of iteration exceeds a certain upper bound**

Cost Function Derivative

- In order to update the parameters in SGD, we need to compute the **partial derivatives** *wrt* the learnable parameters.
- Remember the chain rule:
  - if J is a function of a given $z(x)$, then the derivative of J *wrt* x is:

$$\frac{\partial J}{\partial x} = \frac{\partial J}{\partial z}\frac{\partial z}{\partial x}$$

- Thus (in $R^{2)}$, we need to compute
  - for the i-th example $x^{(i)}$

$$\Delta\vartheta_1 = \frac{\partial}{\partial\theta_1}(h(x^{(i)};\theta,b) - y^{(i)})^2$$

$$\Delta\vartheta_2 = \frac{\partial}{\partial\theta_2}(h(x^{(i)};\theta,b) - y^{(i)})^2$$

$$\Delta b = \frac{\partial}{\partial b}(h(x^{(i)};\theta,b) - y^{(i)})^2$$

Cost Function Derivatives

$$\Delta\theta_1 = \frac{\partial}{\partial\theta_1}(h(x^{(i)};\theta,b) - y^{(i)})^2 =$$

$$= 2((h(x^{(i)};\theta,b) - y^{(i)})\frac{\partial}{\partial\theta_1}(h(x^{(i)};\theta,b))$$

$$= 2(g(\theta^T x^{(i)} + b) - y^{(i)})\frac{\partial}{\partial\theta_1}(g(\theta^T x^{(i)} + b))$$

$$g(z) = \frac{1}{1+e^{-z}}$$

$$\frac{\partial g}{\partial z} = (1 - g(z))g(z)$$

We have that:

$$\frac{\partial}{\partial\theta_1}(g(\theta^T x + b)) = \frac{\partial g(\theta^T x + b)}{\partial(\theta^T x + b)}\frac{\partial(\theta^T x + b)}{\partial\theta_1}$$

$$(1 - g(\theta^T x + b))g(\theta^T x + b)\frac{\partial(\theta_1 x_1 + \theta_2 x_2 + b)}{\partial\theta_1}$$

$$= (1 - g(\theta^T x + b))g(\theta^T x + b)x_1$$

$$s(x) = \frac{1}{1+e^{-x}} \quad \textit{then} \quad \frac{\vartheta s}{\vartheta x} = (1-s(x))s(x)$$

$$\frac{d}{dx}s(x) = \frac{d}{dx}((1+e^{-x})^{-1})$$

$$\frac{d}{dx}s(x) = -1((1+e^{-x})^{(-1-1)})\frac{d}{dx}(1+e^{-x})$$

$$\frac{d}{dx}s(x) = -1((1+e^{-x})^{(-2)})(\frac{d}{dx}(1)+\frac{d}{dx}(e^{-x}))$$

$$\frac{d}{dx}s(x) = -1((1+e^{-x})^{(-2)})(0+e^{-x}(\frac{d}{dx}(-x)))$$

$$\frac{d}{dx}s(x) = -1((1+e^{-x})^{(-2)})(e^{-x})(-1)$$

$$\frac{d}{dx}s(x) = ((1+e^{-x})^{(-2)})(e^{-x})$$

$$\frac{d}{dx}s(x) = \frac{1}{(1+e^{-x})^2}(e^{-x})$$

$$\frac{d}{dx}s(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})}\frac{1}{(1+e^{-x})}$$

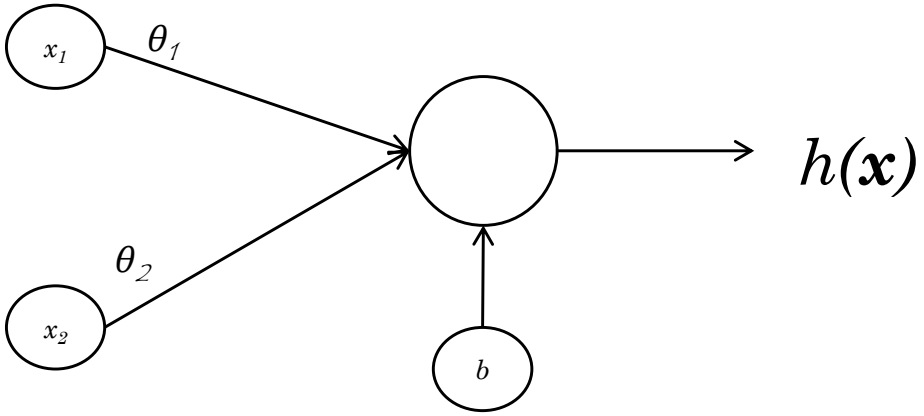
Cost Function Derivatives

Then,

$$\Delta\theta_1 = 2[(g(\theta^T x^{(i)} + b) - y^{(i)})][(1 - g(\theta^T x^{(i)} + b))g(\theta^T x^{(i)} + b)x^{(i)}{}_1]$$

and we can do the same for θ_2

$$\Delta\theta_2 = 2[(g(\theta^T x^{(i)} + b) - y^{(i)})][(1 - g(\theta^T x^{(i)} + b))g(\theta^T x^{(i)} + b)x^{(i)}{}_2]$$



Cost Function Derivatives for *b*

- For the b parameter, the same steps apply:

$$\Delta b = \frac{\partial}{\partial b}(h(x^{(i)};\theta,b) - y^{(i)})^2 =$$

$$= 2((h(x^{(i)};\theta,b) - y^{(i)})\frac{\partial}{\partial b}(h(x^{(i)};\theta,b))$$

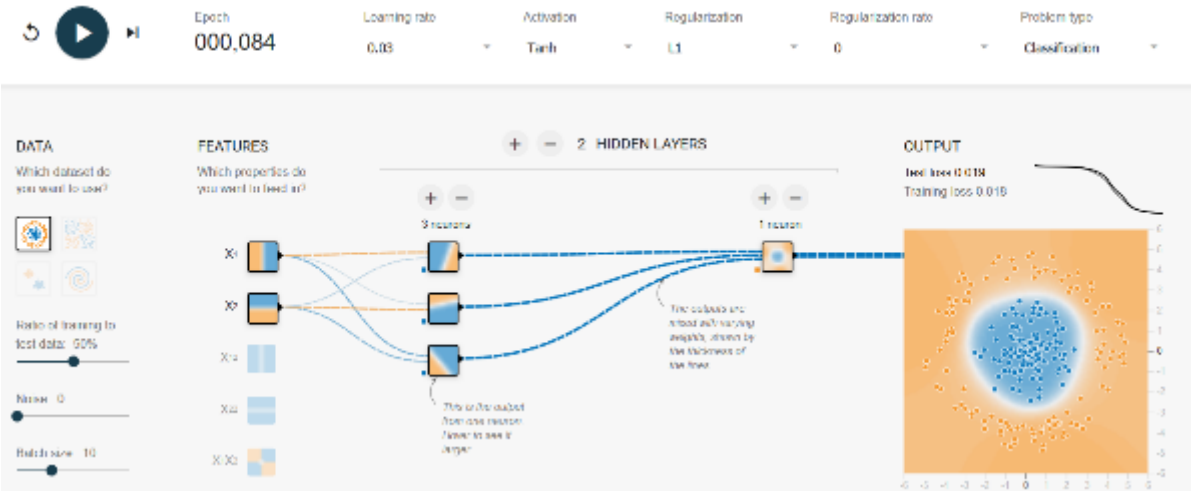
$$= 2(g(\theta^T x^{(i)} + b) - y^{(i)})\frac{\partial}{\partial b}(g(\theta^T x^{(i)} + b))$$

$$\frac{\partial}{\partial b}(g(\theta^T x + b)) = \frac{\partial g(\theta^T x + b)}{\partial(\theta^T x + b)}\frac{\partial(\theta^T x + b)}{\partial b} = (1 - g(\theta^T x + b))g(\theta^T x + b)$$

$$\Delta b = 2[(g(\theta^T x^{(i)} + b) - y^{(i)})][(1 - g(\theta^T x^{(i)} + b))g(\theta^T x^{(i)} + b)]$$

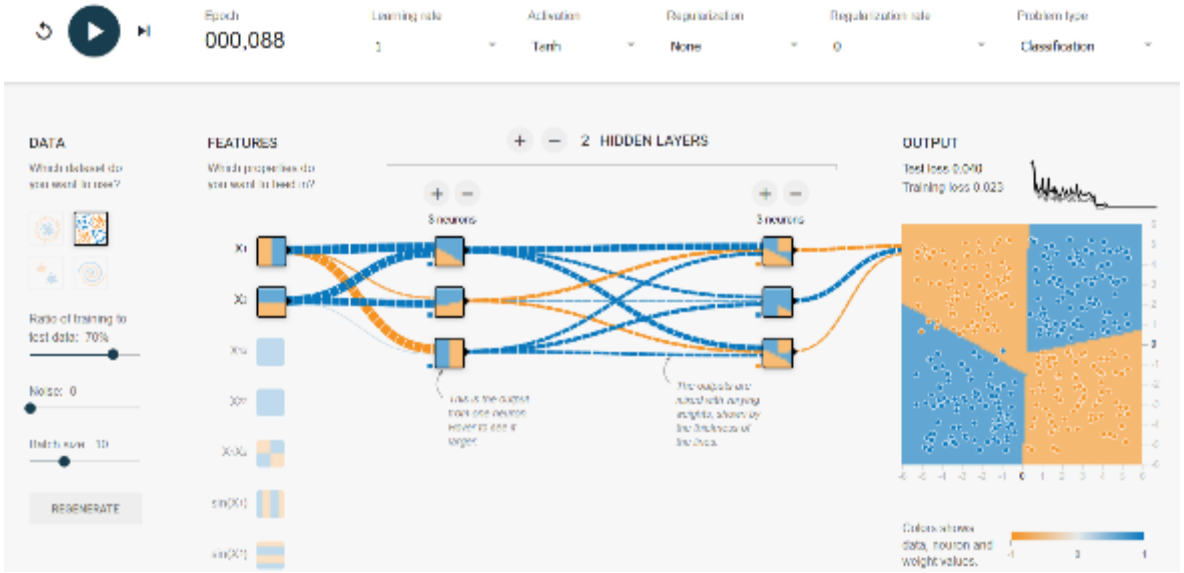
A simple demo on TensorFlow

- Look at: https://playground.tensorflow.org/



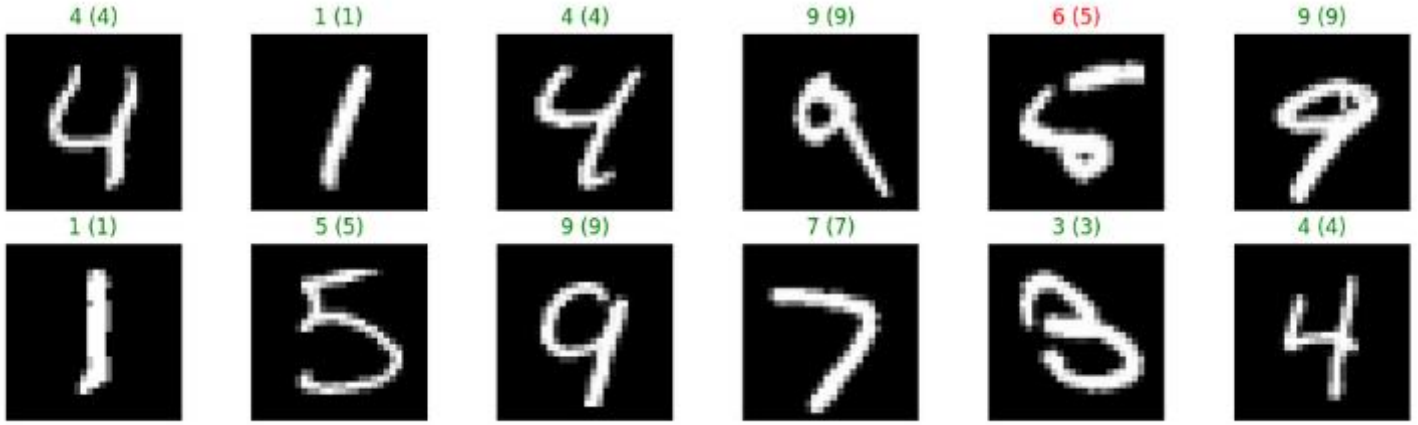
A simple demo on TensorFlow

- Look at: https://playground.tensorflow.org/



The exercise: NNs for MNIST character recognition

- https://colab.research.google.com/drive/1FbsjIx1W5MYAwuy72TcRQA7aMPBqmvYo#scrollTo=OdS2Mx0878tg




```
MLP(
  (fc1): Linear(in_features=784, out_features=20, bias=True)
  (fc2): Linear(in_features=20, out_features=20, bias=True)
  (fc3): Linear(in_features=20, out_features=10, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Riferimenti Bibliografici

- *AIMA*, Chapter 18: 18.16.1, 18.3-4, 18.7

- Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press book, 2016